

算法设计

Design of Algorithms

第2章 递归与分治策略

学习要点

2.1 递归概念。

2.2 分治策略。

2.3 通过下面的范例学习分治策略设计技巧。

(1) 二分搜索技术；

(2) 大整数乘法；

(3) 棋盘覆盖；

(4) 快速排序；

(5) 线性时间选择；

2.1 递归的概念

- **递归**：直接或间接地调用自身的算法称为递归算法。用函数自身给出定义的函数称为递归函数。
- **基本思想**：递归在于把问题分解成规模更小、具有与原来问题相同解法的问题。
- **使用条件**
 - 可以把要解决的问题转化为一个子问题，与原来的解决方法相同，只是问题的规模变小了。递归方程
 - 原问题可以通过子问题解决而组合解决。
 - 存在一种简单的情境，问题在此情境下退出。递归出口，边界条件。
- 由分治法产生的子问题往往是原问题的较小模式，这就为使用递归技术提供了方便。
- 分治与递归像一对孪生兄弟，经常同时应用在算法设计之中，并由此产生许多高效算法。

2.1 递归的概念

➤递归模板:

```
void recursion(int level, int param) {  
    // recursion terminator  
    if (level > MAX_LEVEL) { // 一、递归终结条件, 递归出口  
        // process result  
        return ;  
    }  
  
    // process current logic  
    process(level, param); // 二、处理当前层逻辑  
  
    // drill down  
    recursion(level + 1, param); // 三、下探到下一层  
  
    // reverse the current level status if needed // 四、清理当前层  
}
```

2.1 递归的概念

• 例2.1 阶乘函数

- 阶乘函数可递归地定义为：

$$n! = \begin{cases} 1 & n = 0 \\ n(n-1)! & n > 0 \end{cases}$$

边界条件

递归方程

```
int factorial(int n)
{
    if (n == 0) return 1;
    return n * factorial(n - 1);
}
```

可以找到相应的非递归方式定义：

$$n! = 1 \cdot 2 \cdot 3 \cdots (n-1) \cdot n$$

2.1 递归的概念

• 例2.2 Fibonacci数列

- 无穷数列1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ……，称为Fibonacci数列。它可以递归地定义为：

$$F(n) = \begin{cases} 1 & n = 0 \\ 1 & n = 1 \\ F(n-1) + F(n-2) & n > 1 \end{cases}$$

边界条件

递归方程

第n个Fibonacci数可递归地计算如下：

```
int fibonacci(int n)
```

```
{
```

```
    if (n <= 1) return 1;
```

```
    return fibonacci(n - 1) + fibonacci(n - 2);
```

```
}
```

2.1 递归的概念

例2.4 全排列问题

设计一个递归算法生成 n 个元素 $\{r_1, r_2, \dots, r_n\}$ 的全排列。

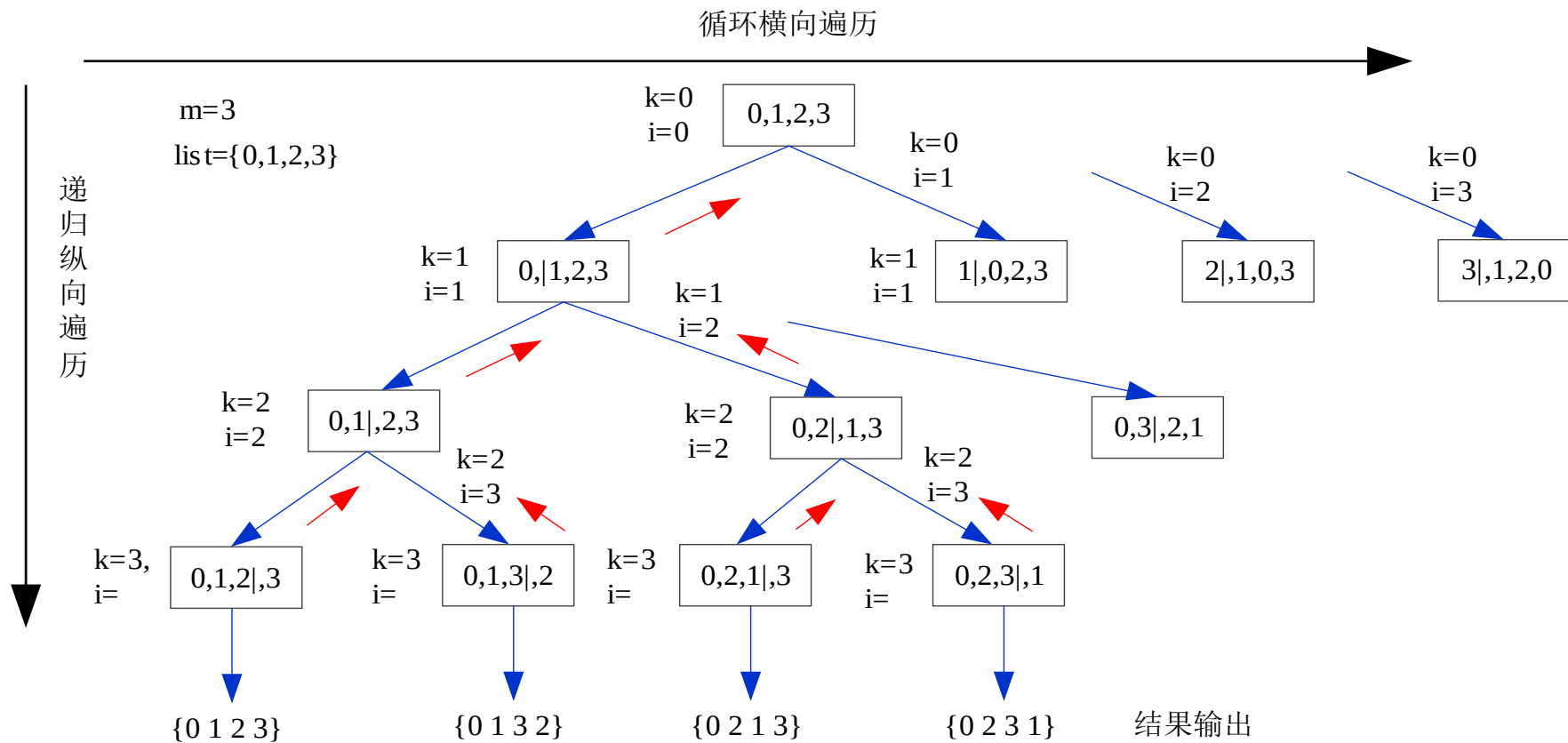
设 $R=\{r_1, r_2, \dots, r_n\}$ 是要进行排列的 n 个元素， $R_i=R-\{r_i\}$ 。集合 X 中元素的全排列记为 $\text{perm}(X)$ 。 $(r_i)\text{perm}(X)$ 表示在全排列 $\text{perm}(X)$ 的每一个排列前加上前缀 r_i 得到的排列。

R 的全排列可归纳定义如下：

- 当 $n=1$ 时， $\text{perm}(R)=(r)$ ，其中 r 是集合 R 中唯一的元素；
- 当 $n>1$ 时， $\text{perm}(R)$ 由 $(r_1)\text{perm}(R_1)$ ， $(r_2)\text{perm}(R_2)$ ， $\dots$ ， $(r_n)\text{perm}(R_n)$ 构成。

2.1 递归的概念

实例 回溯法

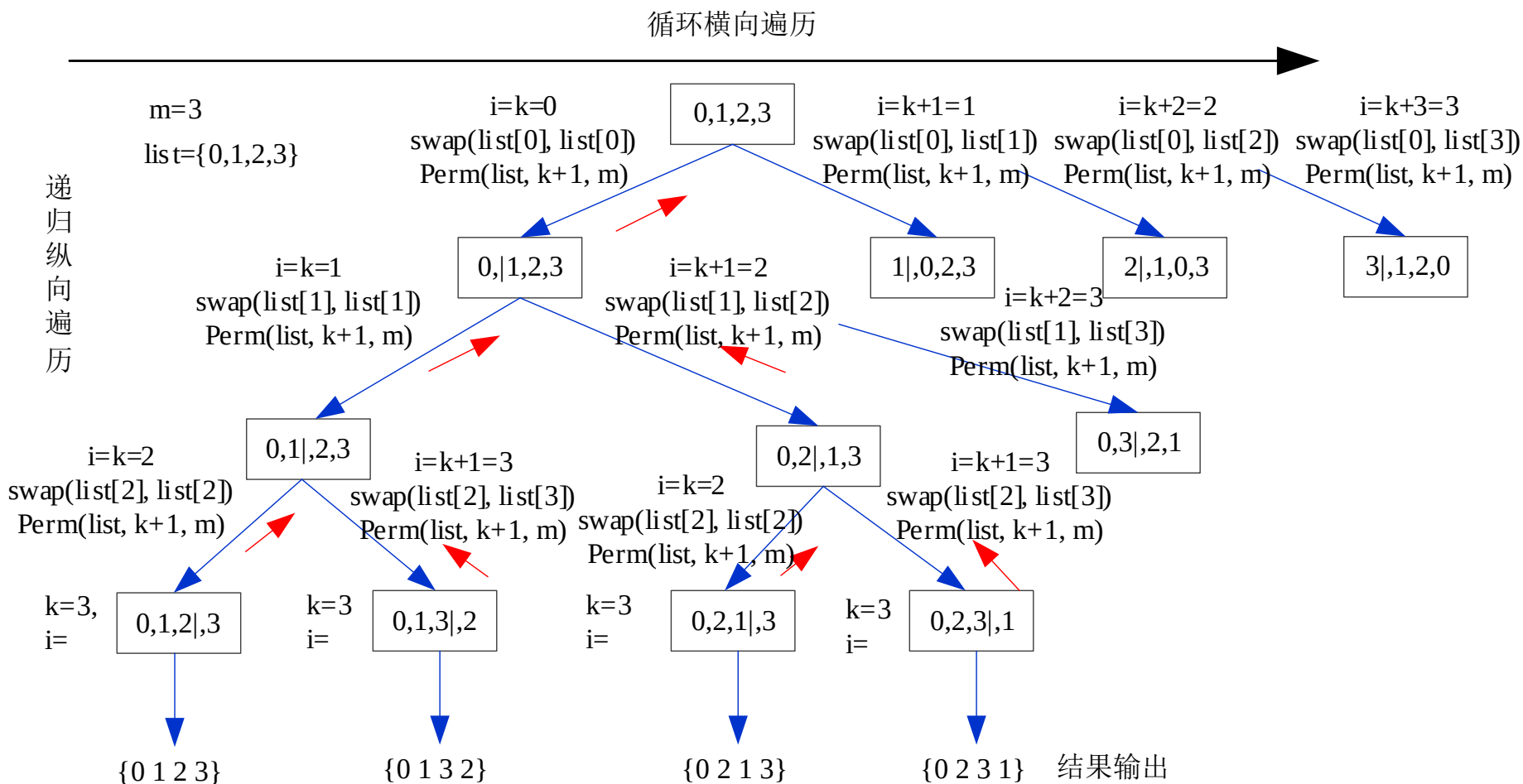


2.1 递归的概念

```
template <class Type>
void Perm(Type list[], int k, int m)  // 产生list[k:m]的所有排列
{
    if (k == m) {  // 只剩下1个元素
        for (int i = 0; i <= m; i++) cout<<list[i];
        cout<<endl;
    } else  // 还有多个元素待排列，递归产生排列
        for (int i = k; i <= m; i++) {
            Swap(list[k], list[i]);
            Perm(list, k+1, m);
            Swap(list[k], list[i]);
        }
}

template <class Type>
inline void Swap(Type &a, Type &b)
{
    Type temp = a; a = b; b = temp;
}
```

2.1 递归的概念



实验2-1：全排列问题

要求见上机报告

2.1 递归的概念

递归小结

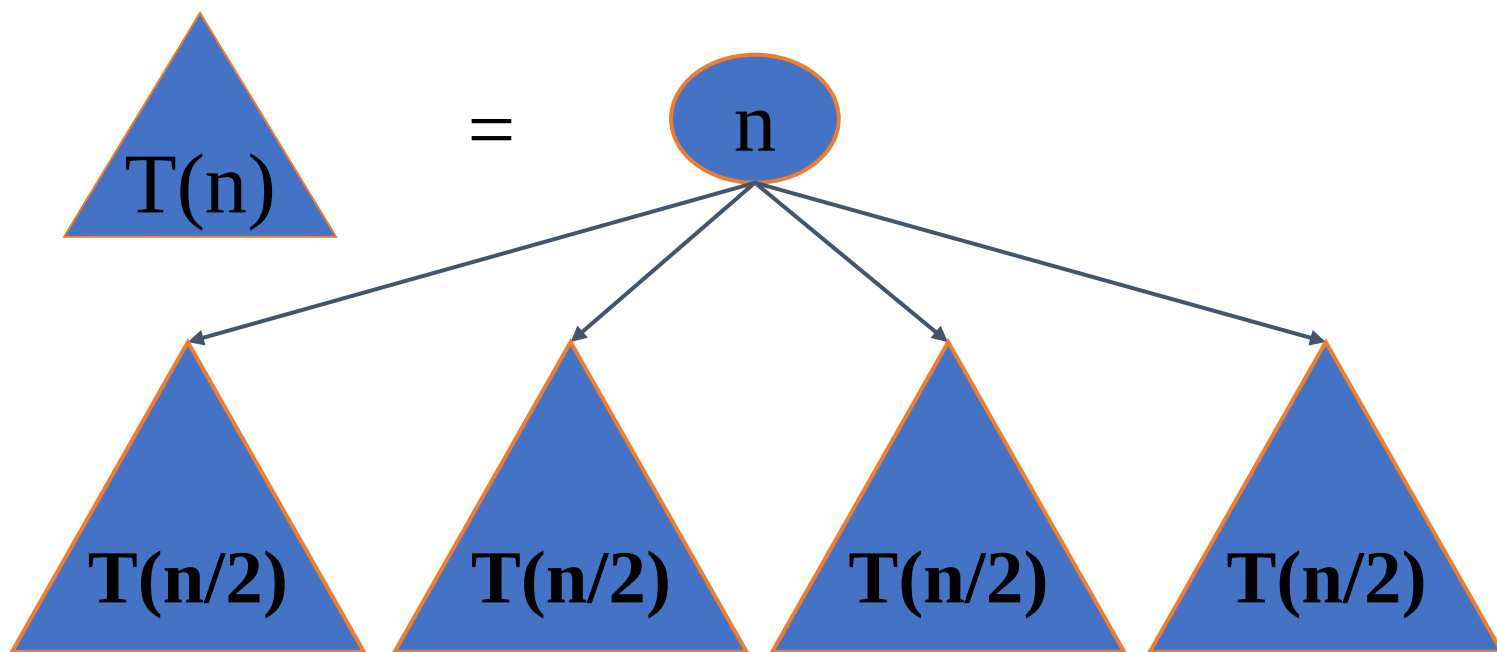
- **优点：** 结构清晰，可读性强，而且容易用数学归纳法来证明算法的正确性，因此它为设计算法、调试程序带来很大方便。
- **缺点：** 递归算法的运行效率较低，无论是耗费的计算时间还是占用的存储空间都比非递归算法要多。

2.2 分治策略

- 思想：所谓“分而治之”就是把一个复杂的算法问题按一定的“分解”方法分为等价的规模较小的若干部分，然后逐个解决，分别找出各部分的解，把各部分的解组成整个问题的解。
- 分治算法，把问题分解为同一性质的子问题，再将子问题分解（递归），直到分解出的问题（最小子问题）可以直接求解。然后由这个解再一层层地回到原问题，同时在此过程中得到对应层的解。
- 步骤：
 - 1.分解（Divide）：将问题分解为同一类型的子问题；
 - 2.治理（Conquer）：递归地解决子问题；
 - 3.合并（Combine）：合并子问题的答案，得出原问题的答案。

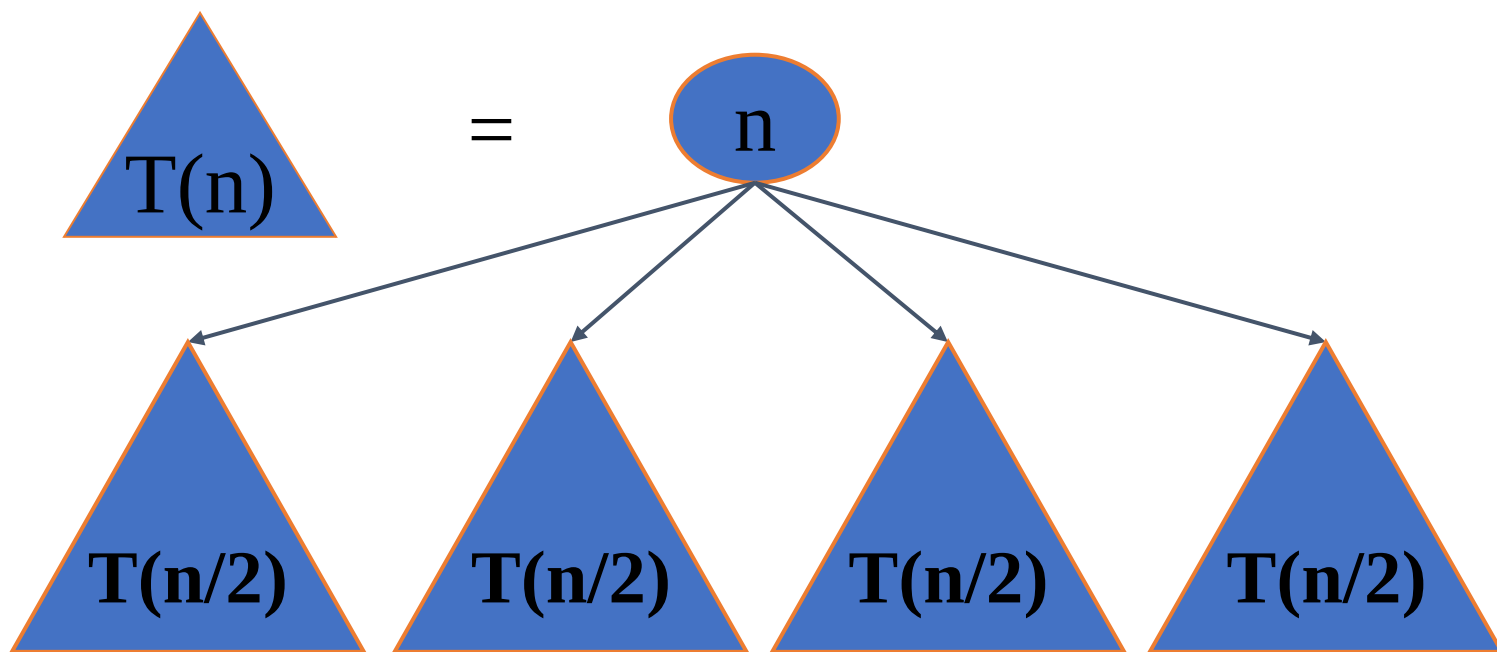
算法总体思想

- 将要求解的较大规模的问题分割成 k 个更小规模的子问题。



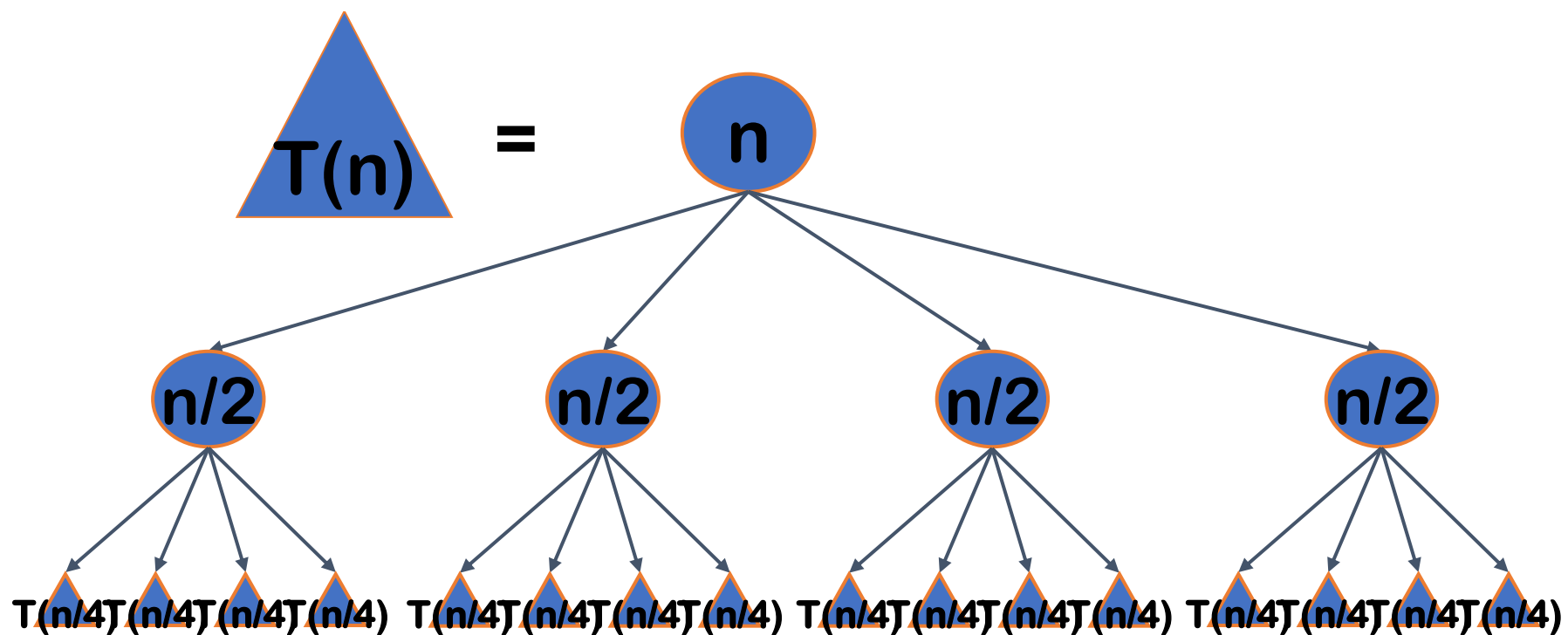
算法总体思想

- 对这 k 个子问题分别求解。如果子问题的规模仍然不够小，则再划分为 k 个子问题，如此递归的进行下去，直到问题规模足够小，很容易求出其解为止。



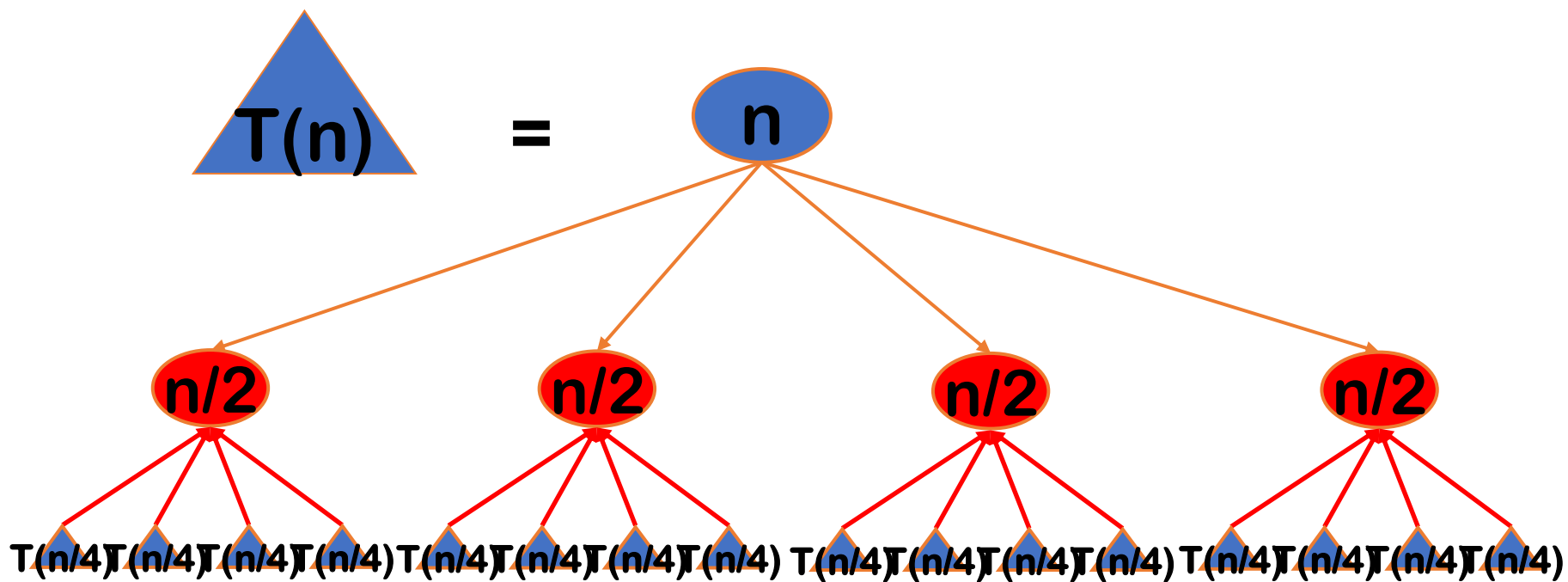
算法总体思想

- 将求出的小规模的问题的解合并为一个更大规模的问题的解，自底向上逐步求出原来问题的解。



算法总体思想

- 将求出的小规模的问题的解合并为一个更大规模的问题的解，自底向上逐步求出原来问题的解。



算法总体思想

- 将求出的小规模的问题的解合并为一个更大规模的问题的解，自底向上逐步求出原来问题的解。

分治法的设计思想是，将一个难以直接解决的大问题，分割成一些规模较小的相同问题，以便各个击破，分而治之。

分治法的适用条件，几个特征：

- 该问题的规模缩小到一定的程度就可以容易地解决；
- 该问题可以分解为若干个规模较小的相同问题，即该问题具有**最优子结构性**；
- 利用该问题分解出的子问题的解可以合并为该问题的解；
- 该问题所分解出的各个子问题是**相互独立**的，即子问题之间不包含公共的子问题。

因为问题的计算复杂性一般是随着问题规模的增加而增加，因此大部分问题满足这个特征。

这条特征是应用分治法的前提，它也是大多数问题可以满足的，此特征反映了递归思想的应用。

能否利用分治法完全取决于问题是否具有这条特征，如果具备了前两条特征，而不具备第三条特征，则可以考虑**贪心算法**或**动态规划**。

这条特征涉及到分治法的效率，如果各子问题是不独立的，则分治法要做许多不必要的工作，重复地解公共的子问题，此时虽然也可用分治法，但一般用**动态规划**较好。

分治法的基本步骤

divide-and-conquer(P)

```
{  
  if ( | P | <= n0) adhoc(P); //解决小规模的问题  
  divide P into smaller subinstances P1,P2,...,Pk; //分解问题  
  for (i=1,i<=k,i++)  
    yi=divide-and-conquer(Pi); //递归的解各子问题  
  return merge(y1,...,yk); //将各子问题的解合并为原问题的解  
}
```

人们从大量实践中发现，在用分治法设计算法时，最好使子问题的规模大致相同。即将一个问题分成大小相等的k个子问题的处理方法是行之有效的。这种使子问题规模大致相等的做法是出自一种**平衡(balancing)**子问题的思想，它几乎总是比子问题规模不等的做法要好。

二分搜索技术

给定已按升序排好序的 n 个元素 $a[0:n-1]$ ，现要在这 n 个元素中找出一特定元素 x 。分析：

- ✓ 该问题的规模缩小到一定的程度就可以容易地解决；
- ✓ 该问题可以分解为若干个规模较小的相同问题；
- ✓ 分解出的子问题的解可以合并为原问题的解；
- ✓ 分解出的各个子问题是相互独立的。

分析：如果 $n=1$ ，即只有一个元素，则只要比较这个元素和 x 就可以确定 x 是否在表中。因此满足分治法的第一个适用条件

分析：比较 x 和 a 的中间元素 $a[mid]$ ，若 $x=a[mid]$ ，则 x 在 L 中的位置就是 mid ；如果 $x<a[mid]$ ，由于递增排序，所以只要在 $a[mid]$ 的前面查找 x ；如果 $x>a[mid]$ ，只要在 $a[mid]$ 的后面查找 x 。在前面还是后面查找 x ，其方法都和 a 中查找 x 一样，只不过是查找的规模缩小了。满足分治法的第二个和第三个适用条件。

分析：很显然此问题分解出的子问题相互独立，即在 $a[mid]$ 的前面或后面查找 x 是独立的子问题，满足分治法的第四个适用条件。

二分搜索技术

给定已按升序排好序的 n 个元素 $a[0:n-1]$ ，现要在这 n 个元素中找出一特定元素 x 。

据此容易设计出二分搜索算法：

```
template<class Type>
int BinarySearch(Type a[], const Type& x, int l, int r)
{
    while (r >= l){
        int m = (l+r)/2;
        if (x == a[m]) return m;
        if (x < a[m]) r = m-1; else l = m+1;
    }
    return -1;
}
```

算法复杂度分析：

每执行一次算法的while循环，待搜索数组的大小减少一半。因此，在最坏情况下，while循环被执行了 $O(\log n)$ 次。循环体内运算需要 $O(1)$ 时间，因此算法在最坏情况下的计算时间复杂度为 $O(\log n)$ 。

大整数的乘法

请设计一个有效的算法，可以进行两个n位十进制大整数的乘法运算

◆小学的方法： $O(n^2)$ ✖效率太低

◆分治法:

Theorem

If $T(n) = aT(\lceil \frac{n}{b} \rceil) + O(n^d)$ (for constants $a > 0, b > 1, d \geq 0$), then:

$$T(n) = \begin{cases} O(n^d) & \text{if } d > \log_b a \\ O(n^d \log n) & \text{if } d = \log_b a \\ O(n^{\log_b a}) & \text{if } d < \log_b a \end{cases}$$

$$X = \begin{array}{|c|c|} \hline a & b \\ \hline \end{array}$$

$$X = a 10^{n/2} + b$$

$$Y = \begin{array}{|c|c|} \hline c & d \\ \hline \end{array}$$

$$Y = c 10^{n/2} + d$$

$$XY = ac 10^n + (ad+bc) 10^{n/2} + bd$$

$$\begin{array}{r} \begin{array}{|c|c|} \hline a & b \\ \hline \end{array} \\ \times \begin{array}{|c|c|} \hline c & d \\ \hline \end{array} \\ \hline \begin{array}{cc} ad & bd \end{array} \end{array}$$

$$\begin{array}{r} + \begin{array}{cc} ac & bc \end{array} \\ \hline ac 10^n + (ad+bc) 10^{n/2} + bd \end{array}$$

- $O(1)$ -divide: $n/2$
- $O(1)$ -ad hoc: 2个一位数相乘
- $4T(n/2)$ -conquer: 4乘法 ac, ad, bc, bd
- $O(n)$ -combine: 2 shifts $10^n, 10^{n/2}$ 3加法

$$T(n) = \begin{cases} O(1) & n = 1 \\ 4T(n/2) + O(n) & n > 1 \end{cases}$$

$$= O(n^2)$$

- 列竖式: $O(n^2)$
- 列树式: divide and conquer 1962, Karatsuba's algorithm

$$\begin{aligned}
 X &= \begin{array}{|c|c|} \hline a & b \\ \hline \end{array} & X &= a 10^{n/2} + b \\
 Y &= \begin{array}{|c|c|} \hline c & d \\ \hline \end{array} & Y &= c 10^{n/2} + d \\
 XY &= ac 10^n + ((a-b)(d-c) + ac + bd) 10^{n/2} + bd
 \end{aligned}$$

$$\begin{array}{r}
 \begin{array}{|c|c|} \hline a & b \\ \hline \end{array} \\
 \times \begin{array}{|c|c|} \hline c & d \\ \hline \end{array} \\
 \hline
 \begin{array}{cc} ac & bd \end{array} \\
 \begin{array}{cc} (a-b)(d-c) & \\ + ac & bd \end{array} \\
 \hline
 \end{array}$$

- $O(1)$ -divide: $n/2$
- $O(1)$ -ad hoc: 2个一位数相乘
- $3T(n/2)$ -conquer: 3乘法 ac , $(a-b)(d-c)$, bd
- $O(n)$ -combine: 2 shifts 10^n , $10^{n/2}$ 6加法

$$T(n) = \begin{cases} O(1) & n = 1 \\ 3T(n/2) + O(n) & n > 1 \end{cases}$$

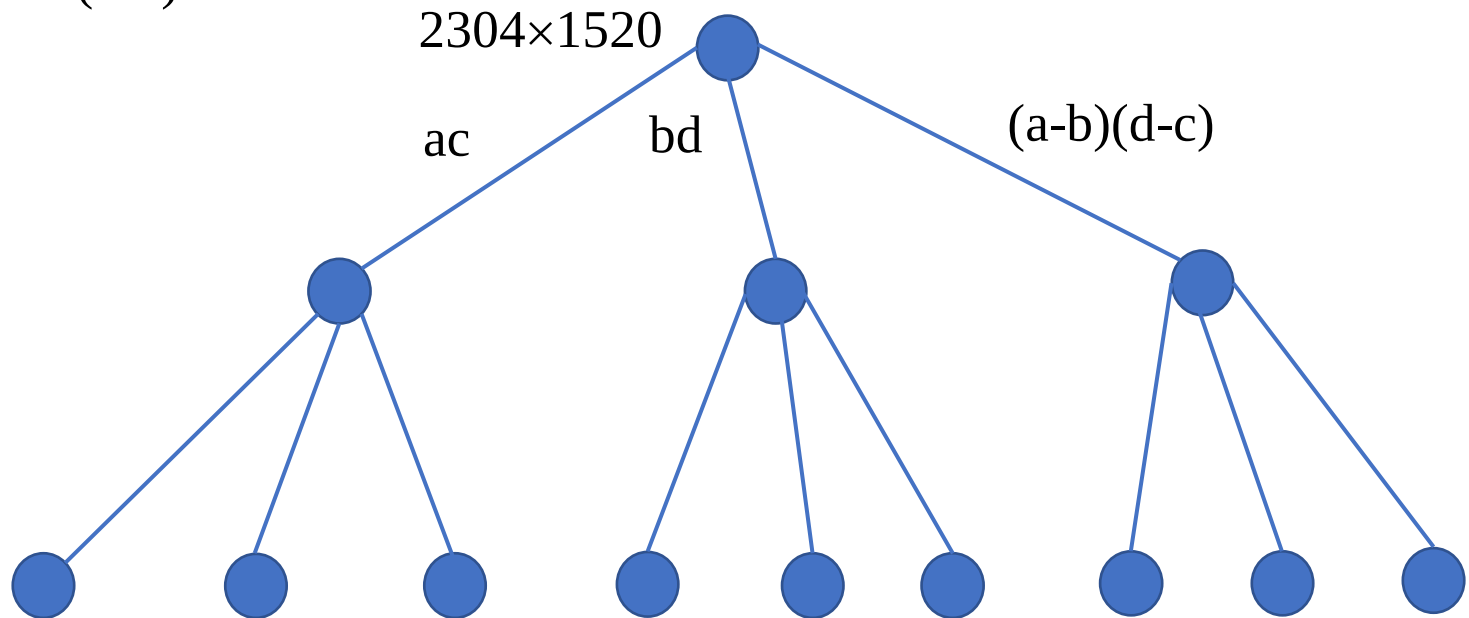
$$ac 10^n + ((a-b)(d-c) + ac + bd) 10^{n/2} + bd = O(n^{\log 3}) = O(n^{1.59})$$

➤ 列树式: divide and conquer 1962, Karatsuba's algorithm

$$XY = \text{ac} 10^n + ((a-b)(d-c) + \text{ac} + \text{bd}) 10^{n/2} + \text{bd}$$

采用分治法计算 2304×1520 ，在递归树的结点处依次标注子问题及其解

1. top-down: $3T(n/2)$ -divide



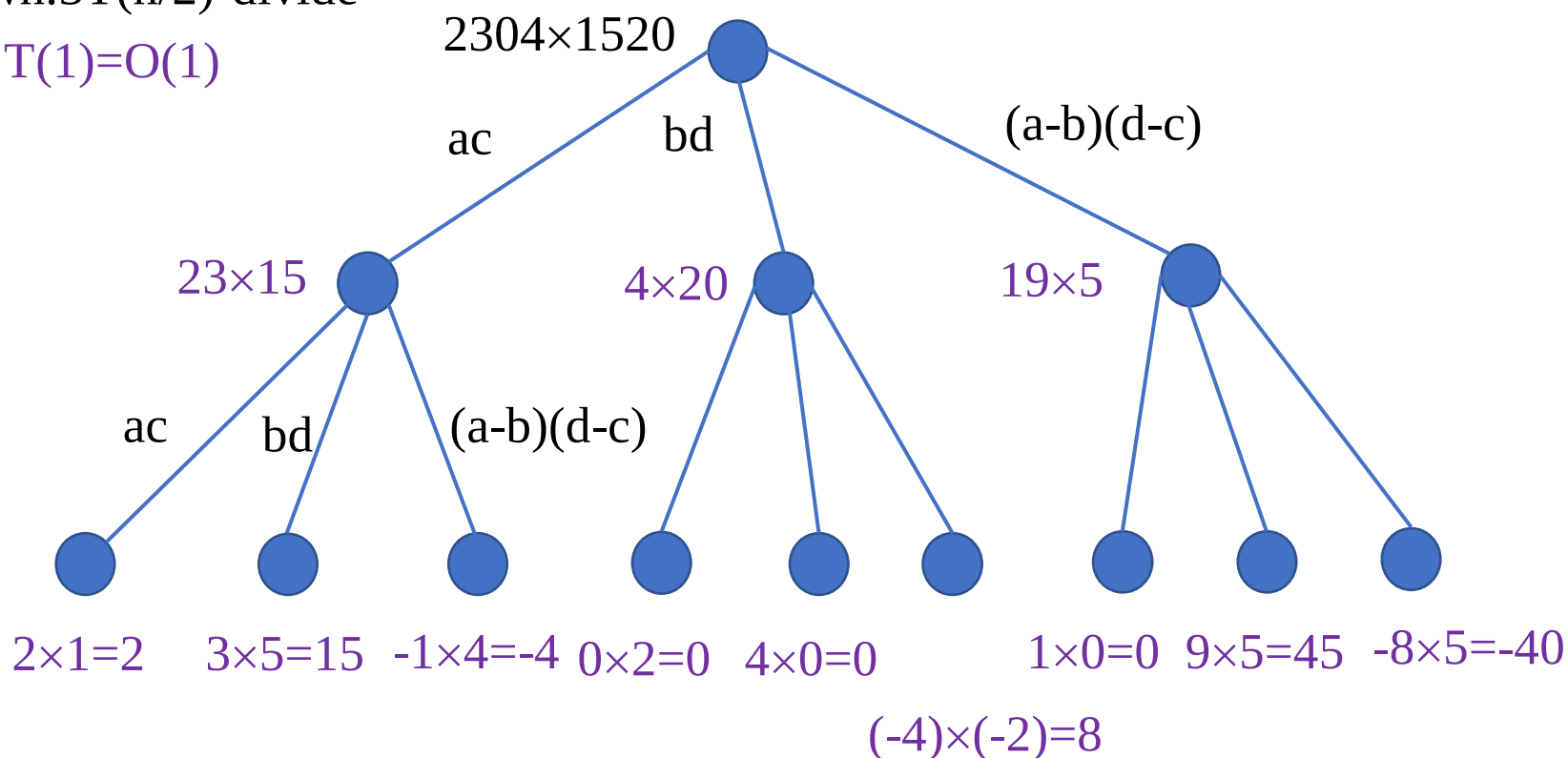
➤ 列树式: divide and conquer 1962, Karatsuba's algorithm

$$XY = \text{ac} 10^n + ((a-b)(d-c) + \text{ac} + \text{bd}) 10^{n/2} + \text{bd}$$

采用分治法计算 2304×1520 , 在递归树的结点处依次标注子问题及其解

1. top-down: $3T(n/2)$ -divide

2. leaves: $T(1) = O(1)$



➤ 列树式: divide and conquer 1962, Karatsuba's algorithm

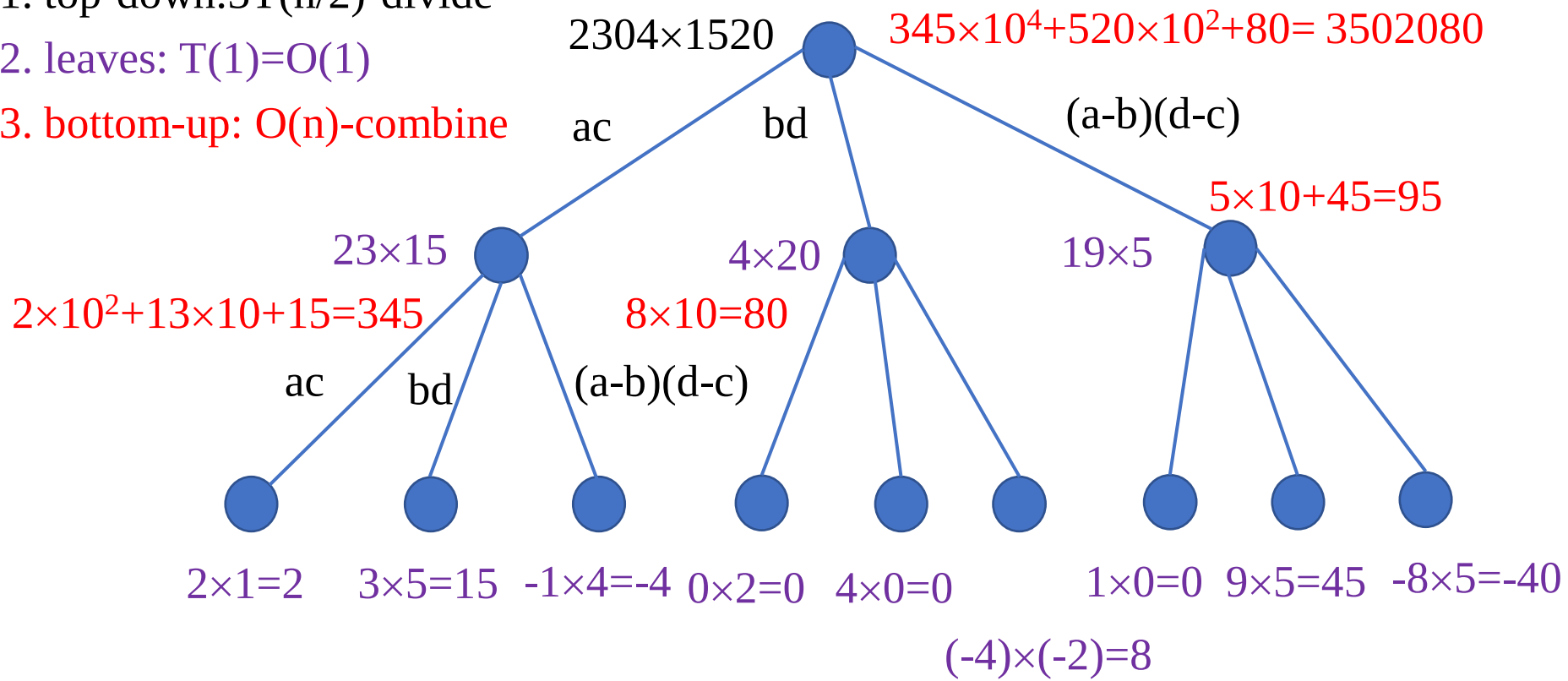
$$XY = \text{ac} 10^n + ((a-b)(d-c) + \text{ac} + \text{bd}) 10^{n/2} + \text{bd}$$

采用分治法计算 2304×1520 , 在递归树的结点处依次标注子问题及其解

1. top-down: $3T(n/2)$ -divide

2. leaves: $T(1) = O(1)$

3. bottom-up: $O(n)$ -combine



大整数的乘法

请设计一个有效的算法，可以进行**两个n位二进制**大整数的乘法运算

◆小学的方法： $O(n^2)$ **✖效率太低**

◆分治法：

$$\begin{array}{l} X = \begin{array}{|c|c|} \hline a & b \\ \hline \end{array} \\ Y = \begin{array}{|c|c|} \hline c & d \\ \hline \end{array} \end{array}$$

$$X = a 2^{n/2} + b \quad Y = c 2^{n/2} + d$$

$$XY = ac 2^n + (ad+bc) 2^{n/2} + bd$$

复杂度分析

$$T(n) = \begin{cases} O(1) & n = 1 \\ 4T(n/2) + O(n) & n > 1 \end{cases}$$

$T(n) = O(n^2)$ **✖没有改进**

大整数的乘法

请设计一个有效的算法，可以进行两个 n 位大整数的乘法运算

◆小学的方法： $O(n^2)$

✖效率太低

◆分治法:

$$XY = ac \cdot 2^n + (ad+bc) \cdot 2^{n/2} + bd$$

为了降低时间复杂度，必须减少乘法的次数。

$$XY = ac \cdot 2^n + ((a-b)(d-c)+ac+bd) \cdot 2^{n/2} + bd$$

复杂度分析

$$T(n) = \begin{cases} O(1) & n = 1 \\ 3T(n/2) + O(n) & n > 1 \end{cases}$$

$$T(n) = O(n^{\log_3 3}) = O(n^{1.59}) \quad \checkmark \text{较大的改进}$$

➤如果将大整数分成更多段，用更复杂的方式把它们组合起来，将有可能得到更优的算法。

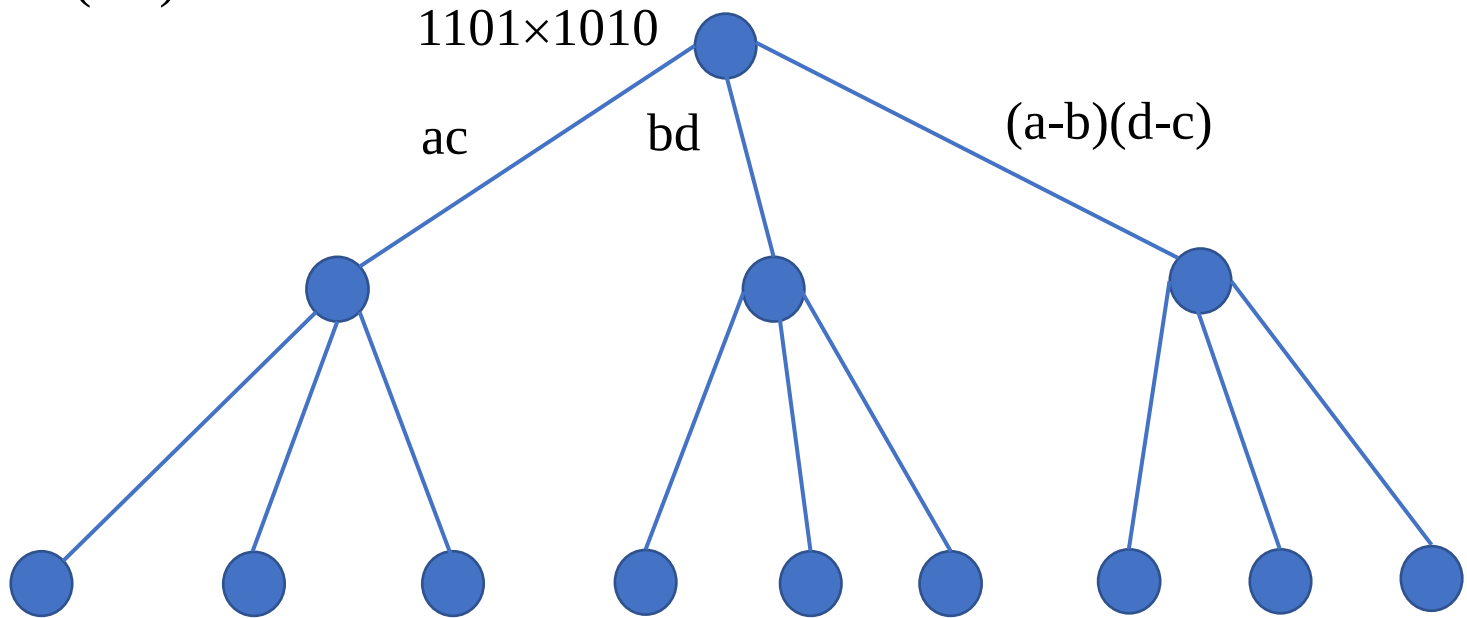
➤最终的，这个思想导致了快速傅利叶变换(Fast Fourier Transform)的产生。该方法也可以看作是一个复杂的分治算法。

练习2.1：大整数乘法

$$XY = ac \cdot 2^n + ((a-b)(d-c) + ac + bd) \cdot 2^{n/2} + bd$$

要求：采用分治法计算二进制数 1101×1010 ，在递归树的结点处依次标注子问题及其解。

1. top-down: $3T(n/2)$ -divide



实验2-3：大整数乘法问题

请设计一个有效的算法，可以进行两个 n 位大整数的乘法运算。

要求见上机报告

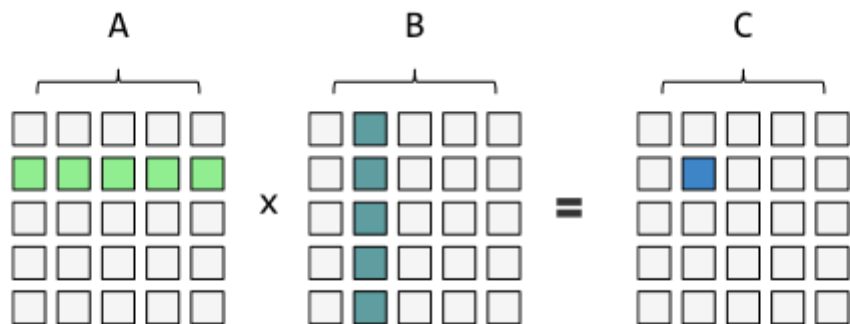
算法实例见上例

Strassen矩阵乘法

◆传统方法： $O(n^3)$

设A和B是两个 $n \times n$ 矩阵

A和B的乘积矩阵C中的元素 $C[i,j]$ 定义为：
$$C[i][j] = \sum_{k=1}^n A[i][k]B[k][j]$$



$$C[i][j] = \text{sum}(A[i][k] * B[k][j]) \text{ for } k = 0 \dots n$$

若依此定义来计算A和B的乘积矩阵C，则每计算C的一个元素 $C[i][j]$ ，需要做 n 次乘法和 $n-1$ 次加法。因此，算出矩阵C的 n^2 个元素所需的计算时间为 $O(n^3)$

Strassen矩阵乘法

◆传统方法: $O(n^3)$

◆分治法:

使用与上例类似的技术, 将矩阵A, B和C中每一矩阵都分块成4个大小相等的子矩阵。由此可将方程 $C=AB$ 重写为:

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

由此可得:

$$C_{11} = A_{11}B_{11} + A_{12}B_{21}$$

$$C_{12} = A_{11}B_{12} + A_{12}B_{22}$$

$$C_{21} = A_{21}B_{11} + A_{22}B_{21}$$

$$C_{22} = A_{21}B_{12} + A_{22}B_{22}$$

复杂度分析

$$T(n) = \begin{cases} O(1) & n = 2 \\ 8T(n/2) + O(n^2) & n > 2 \end{cases}$$

$T(n)=O(n^3)$

Strassen矩阵乘法

◆传统方法: $O(n^3)$

◆分治法:

为了降低时间复杂度, 必须减少乘法的次数。

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

$$M_1 = A_{11}(B_{12} - B_{22})$$

$$M_2 = (A_{11} + A_{12})B_{22}$$

$$M_3 = (A_{21} + A_{22})B_{11}$$

$$M_4 = A_{22}(B_{21} - B_{11})$$

$$M_5 = (A_{11} + A_{22})(B_{11} + B_{22})$$

$$M_6 = (A_{12} - A_{22})(B_{21} + B_{22})$$

$$M_7 = (A_{11} - A_{21})(B_{11} + B_{12})$$



$$C_{11} = M_5 + M_4 - M_2 + M_6$$

$$C_{12} = M_1 + M_2$$

$$C_{21} = M_3 + M_4$$

$$C_{22} = M_5 + M_1 - M_3 - M_7$$

复杂度分析

$$T(n) = \begin{cases} O(1) & n = 2 \\ 7T(n/2) + O(n^2) & n > 2 \end{cases}$$

$$T(n) = O(n^{\log_2 7}) = O(n^{2.81}) \checkmark \text{较大的改进}$$

Strassen矩阵乘法

- ◆传统方法: $O(n^3)$
- ◆分治法: $O(n^{2.81})$
- ◆更快的方法??

➤Hopcroft和Kerr已经证明(1971), 计算2个 2×2 矩阵的乘积, 7次乘法是必要的。因此, 要想进一步改进矩阵乘法的时间复杂性, 就不能再基于计算 2×2 矩阵的7次乘法这样的方法了。或许应当研究 3×3 或 5×5 矩阵的更好算法。

➤在Strassen之后又有许多算法改进了矩阵乘法的计算时间复杂性。目前最好的计算时间上界是 $O(n^{2.376})$

➤是否能找到 $O(n^2)$ 的算法?

棋盘覆盖

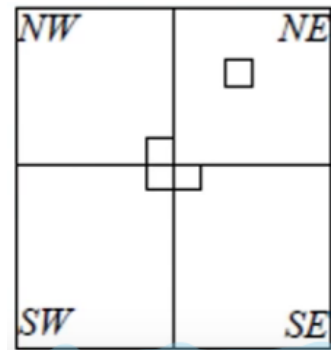
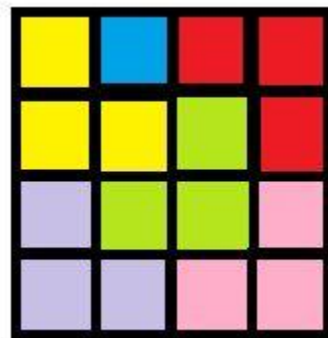
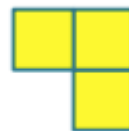
➤ **定义：** 在一个 $2^k \times 2^k$ 个方格组成的棋盘中，恰有一个方格与其它方格不同，称为特殊方格，且称该棋盘为一特殊棋盘。在棋盘覆盖问题中，要用图示的4种不同形态的L型骨牌覆盖给定的特殊棋盘上除特殊方格以外的所有方格，且任何2个L型骨牌不得重叠覆盖。

在 $2^k \times 2^k$ 的棋盘覆盖中，用到的L型骨牌数恰为 $(4^k - 1)/3$ ，即（所有方格个数 - 特殊方格个数）/3。

➤ **实例：** 如图，为 $k=2$ 时的一个特殊棋盘（相同颜色的三个小方格组成一个L型骨牌）和4种不同形态的L型骨牌，蓝色的为特殊方格。

➤ **证明有解：数学归纳法**

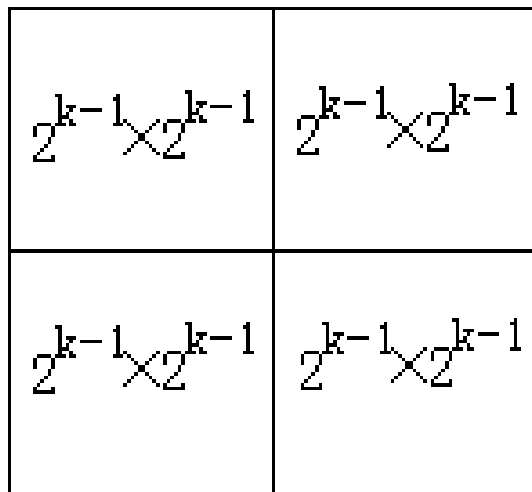
- 当 $n=1$ （ 2×2 棋盘），该问题有解；
- 假设当 $n=k$ 时（ $2^k \times 2^k$ 棋盘），该问题有解
- 那么当 $n=k+1$ 时（ $2^{k+1} \times 2^{k+1}$ 棋盘），将棋盘划分为4个 $2^k \times 2^k$ 子棋盘，特殊方格位于4个子棋盘之一中，而其他3个子棋盘中无特殊方格。
- 用一个L型骨牌覆盖这3个较小棋盘的会合处，将原问题转化为4个 $n=k$ 时的子问题，因为 $n=k$ 时有解，所以 $n=k+1$ 时也有解。



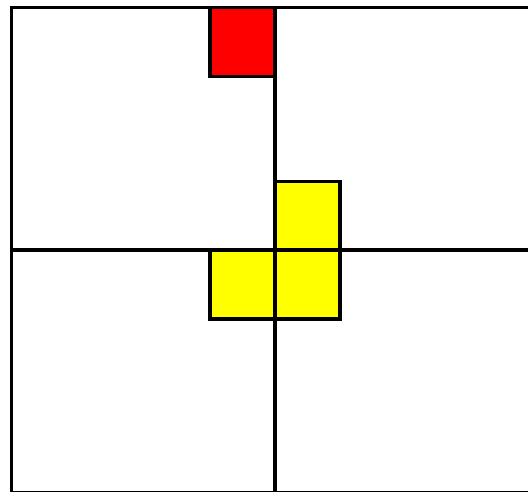
棋盘覆盖

分治法的基本思路：

- 当 $k > 0$ 时，将 $2^k \times 2^k$ 棋盘分割为4个 $2^{k-1} \times 2^{k-1}$ 子棋盘(a)所示。特殊方格必位于4个较小子棋盘之一中，其余3个子棋盘中无特殊方格。
- 为了将这3个无特殊方格的子棋盘转化为特殊棋盘，可以用一个L型骨牌覆盖这3个较小棋盘的会合处，如(b)所示，从而将原问题转化为4个较小规模的棋盘覆盖问题。
- 递归地使用这种分割，直至棋盘简化为棋盘 1×1 。

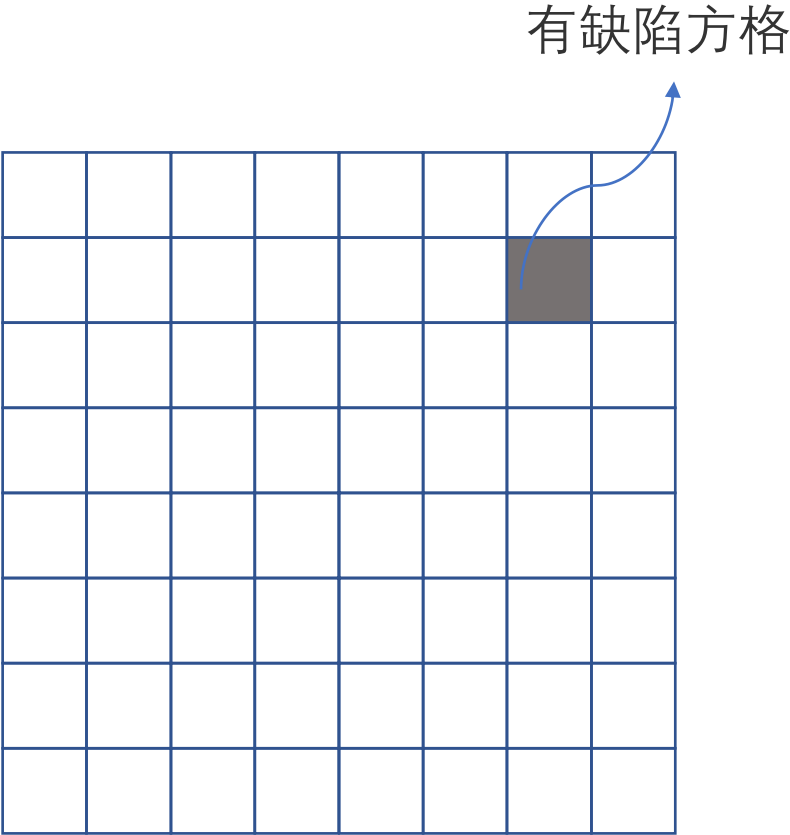


(a)

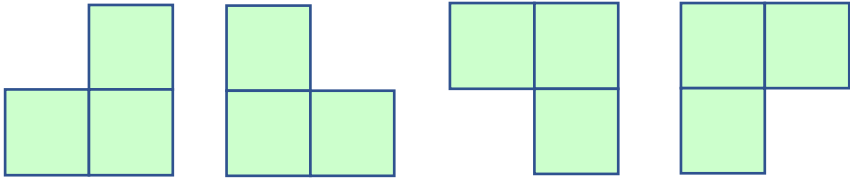


(b)

棋盘覆盖

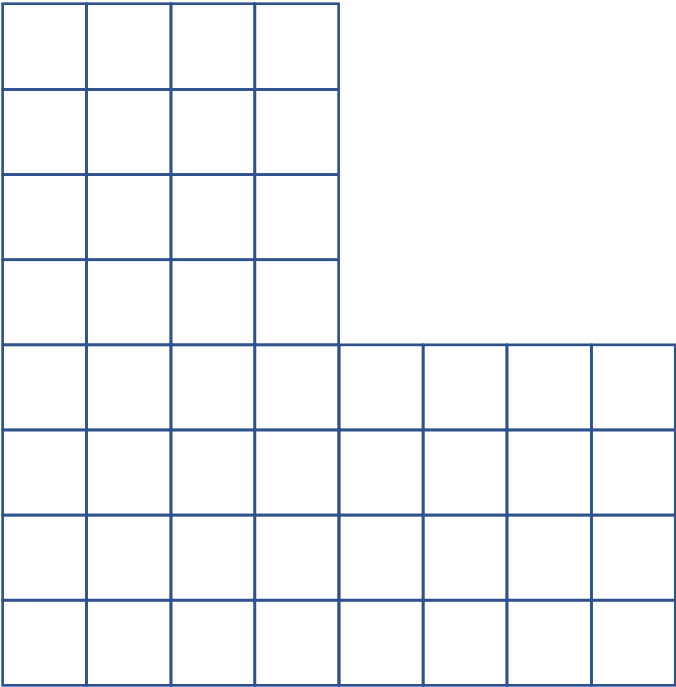


$2^k \times 2^k$ 棋盘

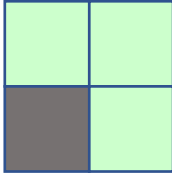
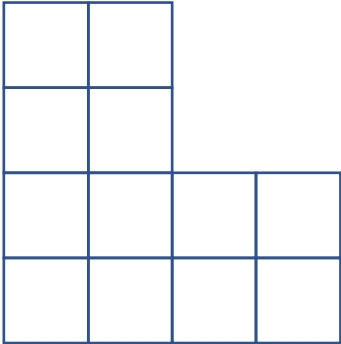


L型3骨牌

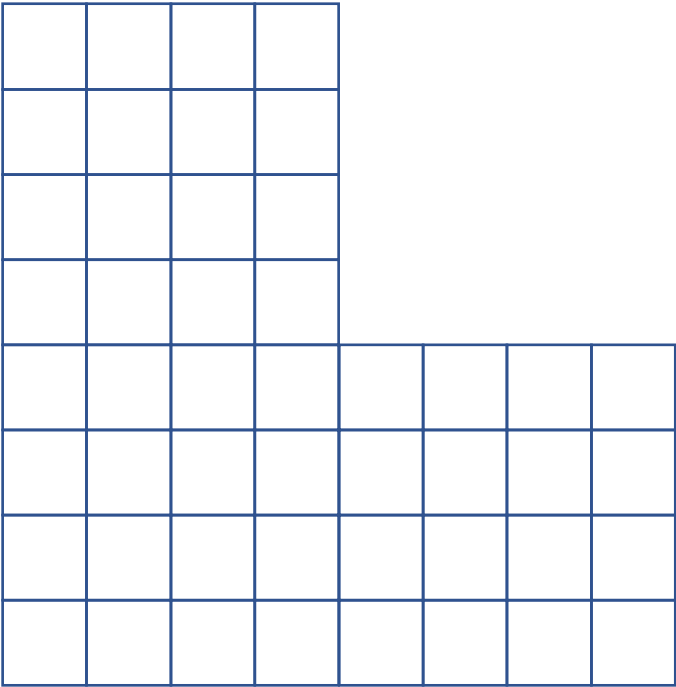
棋盘覆盖



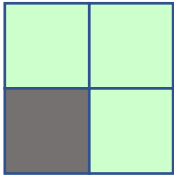
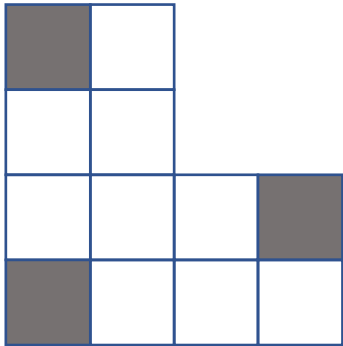
$2^k \times 2^k$ 棋盘



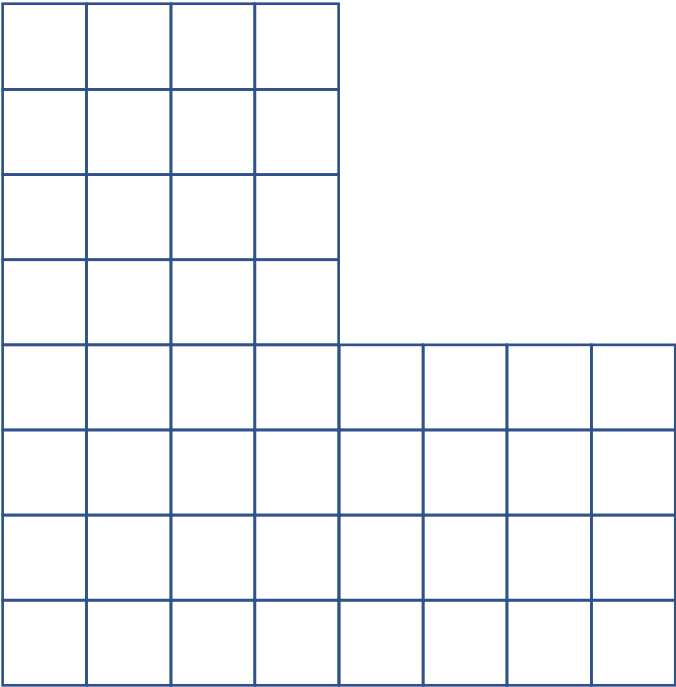
棋盘覆盖



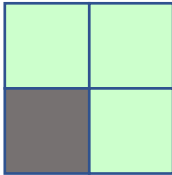
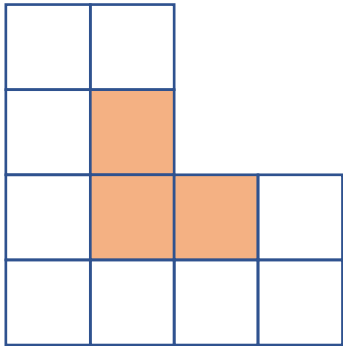
$2^k \times 2^k$ 棋盘



棋盘覆盖

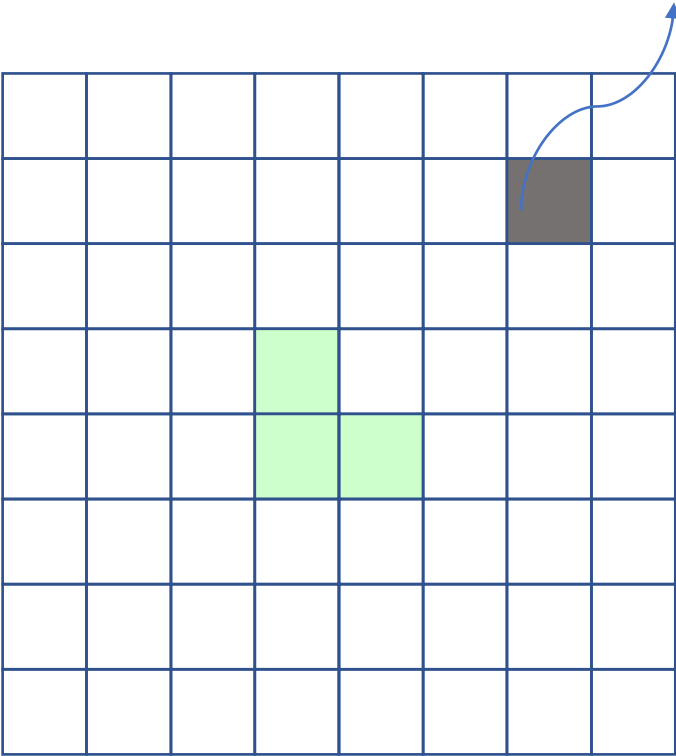


$2^k \times 2^k$ 棋盘

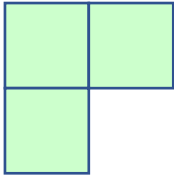
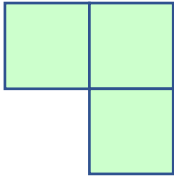
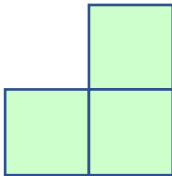


棋盘覆盖

有缺陷方格

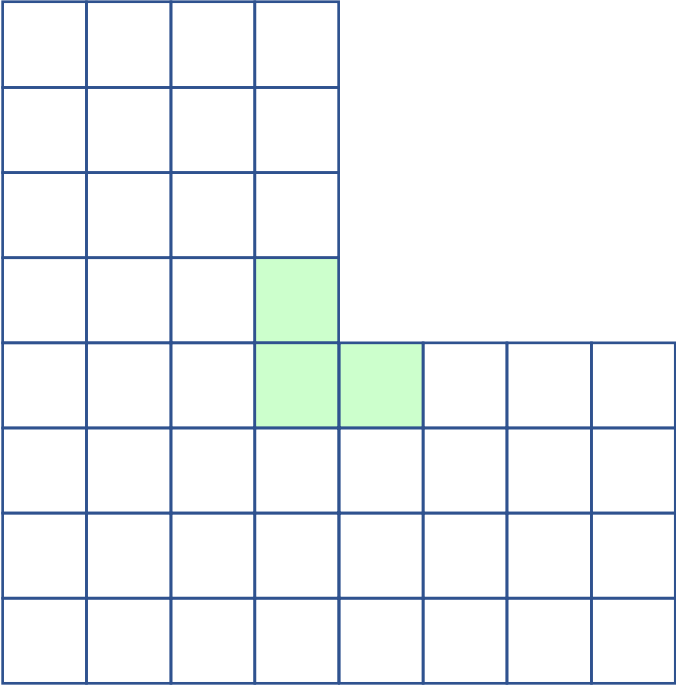


$2^k \times 2^k$ 棋盘

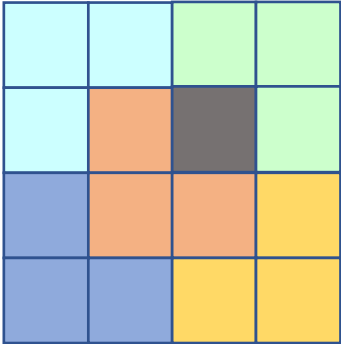


L型3骨牌

棋盘覆盖



$2^k \times 2^k$ 棋盘



棋盘覆盖

chessBoard((tr,tc,dr,dc,size), size= 2^k)

board[tr][tc]

如果残缺方格在左上角棋盘
中则递归处理该残缺子盘
(直到1-by-1子棋盘为止)

否则

在左上角子棋盘的右下角
方格写入t递归处理该残
缺子棋盘(直到1-by-1子棋
盘为止)

如果残缺方格在左下角棋盘
中则递归处理该残缺子盘
(直到1-by-1子棋盘为止)

否则

在左下角子棋盘的右上角
方格写入t递归处理该残
缺子棋盘(直到1-by-1子棋
盘为止)

s=size/2

			t

s=size/2

			t

t			

如果残缺方格在右上角棋盘
中则递归处理该残缺子盘
(直到1-by-1子棋盘为止)

否则

在右上角子棋盘的左下角
方格写入t递归处理该残
缺子棋盘(直到1-by-1子棋
盘为止)

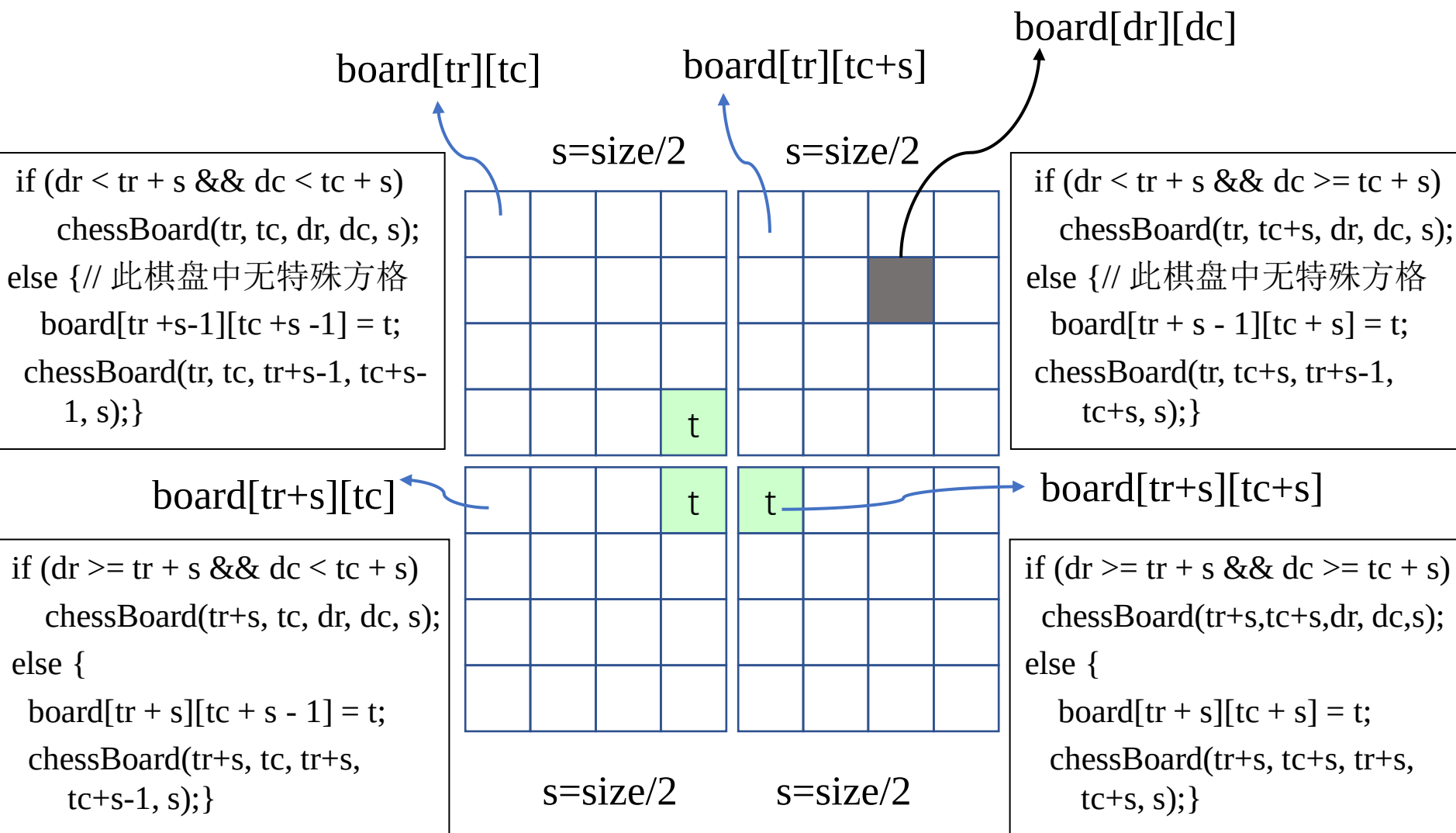
如果残缺方格在右下角棋盘
中则递归处理该残缺子盘
(直到1-by-1子棋盘为止)

否则

在右下角子棋盘的左上角
方格写入t递归处理该残
缺子棋盘(直到1-by-1子棋
盘为止)

棋盘覆盖

chessBoard((tr,tc,dr,dc,size), size= 2^k)



棋盘覆盖

```
void chessBoard(int tr, int tc, int dr, int dc, int size)
```

```
{  
    if (size == 1) return;  
    int t = tile++, // L型骨牌号  
        s = size/2; // 分割棋盘  
    // 覆盖左上角子棋盘  
    if (dr < tr + s && dc < tc + s)  
        // 特殊方格在此棋盘中  
        chessBoard(tr, tc, dr, dc, s);  
    else { // 此棋盘中无特殊方格  
        // 用 t 号L型骨牌覆盖右下角  
        board[tr + s - 1][tc + s - 1] = t;  
        // 覆盖其余方格  
        chessBoard(tr, tc, tr+s-1, tc+s-1, s);  
    }  
    // 覆盖右上角子棋盘  
    if (_____  
        // 特殊方格在此棋盘中  
        chessBoard(_____  
    else { // 此棋盘中无特殊方格  
        // 用 t 号L型骨牌覆盖左下角
```

```
board[tr + s - 1][tc + s] = t;
```

```
// 覆盖其余方格
```

```
chessBoard(_____)};
```

```
// 覆盖左下角子棋盘
```

```
// 覆盖右下角子棋盘
```

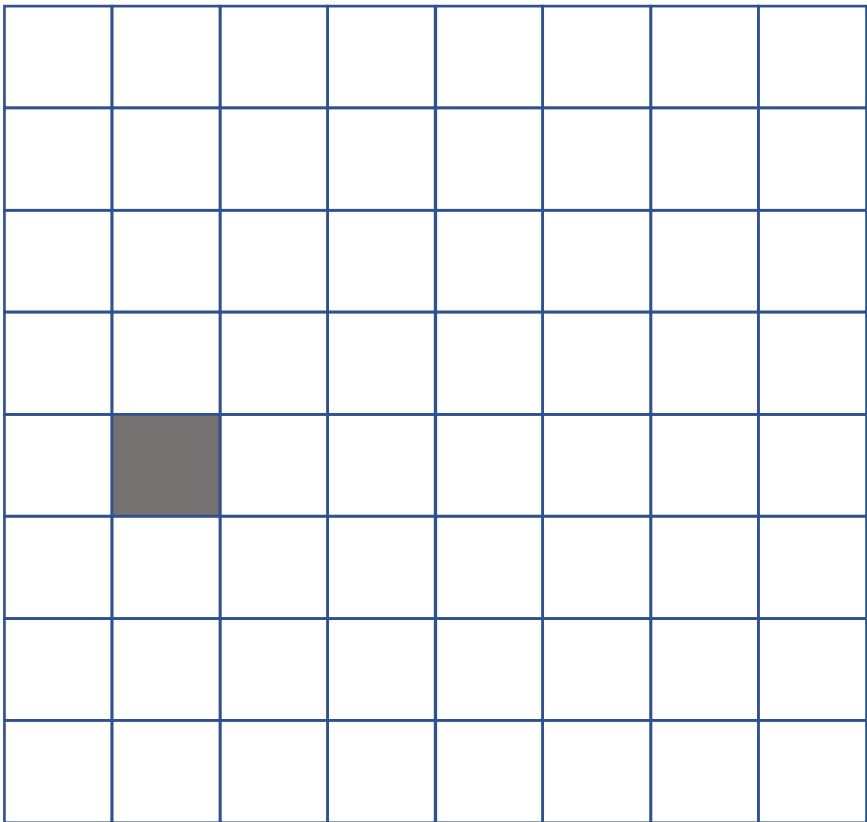
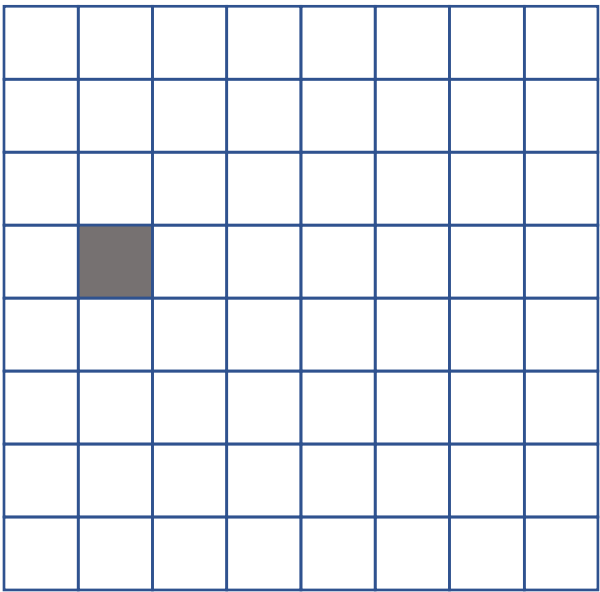
```
}
```

复杂度分析

$$T(k) = \begin{cases} O(1) & k = 0 \\ 4T(k-1) + O(1) & k > 0 \end{cases}$$

$T(n) = O(4^k)$ 渐近意义下的最优算法

练习2.2： 棋盘覆盖。 给定8*8棋盘及缺陷方格位置， 要求给出L型骨牌填充次序。 左侧为示例， 完成右侧填充。



```
请输入棋盘的行列号8
请输入特殊方格的行列号4 2
 2   2   3   3   7   7   8   8
 2   1   1   3   7   6   6   8
 5   5   1   4  10  10   6   9
 5   0   4   4   0  10   9   9
17  17  18   0   0  12  13  13
17  16  18  18  12  12  11  13
20  16  16  19  15  11  11  14
20  20  19  19  15  15  14  14
请按任意键继续. . .
```

2.8 快速排序

```
template<class Type>
void QuickSort (Type a[], int p, int r)
{
    if (p<r) {
        int q=Partition(a, p, r);
        QuickSort (a, p, q-1); //对左半段排序
        QuickSort (a, q+1, r); //对右半段排序
    }
}
```

在快速排序中，记录的比较和交换是从两端向中间进行的，关键字较大的记录一次就能交换到后面单元，关键字较小的记录一次就能交换到前面单元，记录每次移动的距离较大，因而总的比较和移动次数较少。

2.8 快速排序

```
template<class Type>
int Partition (Type a[], int p, int r)
{
    int i = p, j = r + 1;
    Type x=a[p];// a[p] pivot枢轴
    // 将< x的元素交换到左边区域
    // 将> x的元素交换到右边区域
    while (true) {
        while (a[++i] < x);
        while (a[--j] > x);
        if (i >= j) break;
        Swap(a[i], a[j]);
    }
    a[p] = a[j];
    a[j] = x;
    return j;
}
```

{ 6, 7, 5, 2, 5, 8} 初始序列

{ 6, 7, 5, 2, 5, 8} j--;

{ 5, 7, 5, 2, 6, 8} i++;

{ 5, 6, 5, 2, 7, 8} j--;

{ 5, 2, 5, 6, 7, 8} i++;

{ 5, 2, 5} 6 {7, 8} 完成

2.8 快速排序

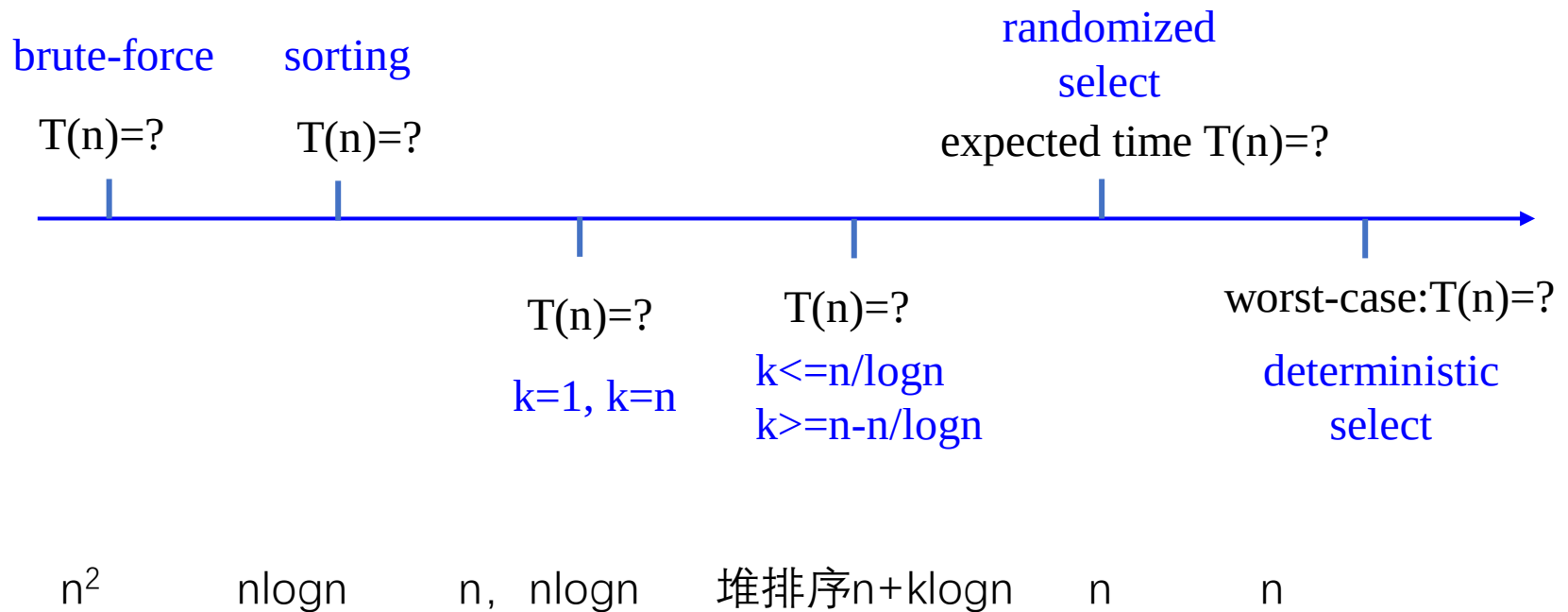
- 快速排序算法的性能取决于划分的对称性。
- 通过修改**partition**，可以优化快速排序算法。
- 随机选择策略：当数组还没有被划分时，可以在 $a[p:r]$ 中随机选出一个元素作为划分基准，这样可以使划分基准的选择是随机的，从而可以期望划分是较对称的。

```
template<class Type>
int RandomizedPartition (Type a[], int p, int r)
{
    int i = Random(p,r);
    Swap(a[i], a[p]);
    return Partition (a, p, r);
}
```

- 最坏时间复杂度： $O(n^2)$ 随机化后： $1.38n\log n$
- 平均时间复杂度： $O(n\log n)$

2.9 线性时间选择

给定线性序集中 n 个元素和一个整数 k , $1 \leq k \leq n$, 要求找出这 n 个元素中第 k 小的元素。



2.9 线性时间选择

给定线性序集中 n 个元素和一个整数 k , $1 \leq k \leq n$, 要求找出这 n 个元素中第 k 小的元素。

```
template<class Type>
```

```
Type RandomizedSelect(Type a[], int p, int r, int k)
```

```
{
```

```
    if (p==r) return a[p];
```

```
    int s=RandomizedPartition(a, p, r),
```

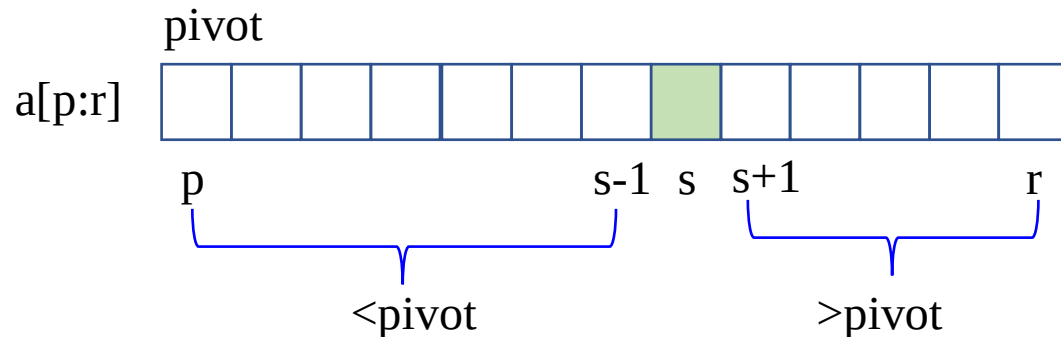
```
    j=s-p+1;
```

```
    if (k<=j) return RandomizedSelect(a, p, s, k);
```

```
    else return RandomizedSelect(a, s+1, r, k-j);
```

```
}
```

- 快排算法划分过程中, pivot 左侧都比pivot值小。
- 通过对划分的两部分计数可以找到第 k 小的元素。



在最坏情况下, 算法**randomizedSelect**需要 $O(n^2)$ 计算时间

算法**randomizedSelect**可以在 $O(n)$ 平均时间内找出 n 个输入元素中的第 k 小元素。

2.9 线性时间选择

如果能在线性时间内找到一个划分基准, 使得按这个基准所划分出的2个子数组的长度都至少为原数组长度的 ε 倍($0 < \varepsilon < 1$ 是某个正常数), 那么就可以在**最坏情况下**用 $O(n)$ 时间完成选择任务。

例如: 若 $\varepsilon=9/10$, 算法递归调用所产生的子数组的长度至少缩短 $1/10$ 。所以, 在最坏情况下, 算法所需的计算时间 $T(n)$ 满足递归式 $T(n) \leq T(9n/10) + O(n)$ 。由此可得 $T(n) = O(n)$ 。

Divide and Conquer:

base case: $n < n_0$, sorting

divide: $O(n)$ -MoM-partition

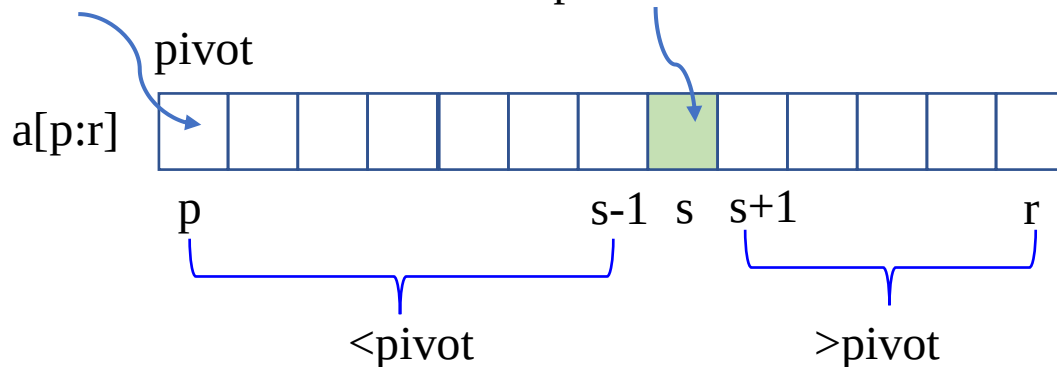
conquer: $T(s-p)$ or $T(r-s)$

combine: zero

Median of Medians, MoM

中位数数组的中位数

pivot所在位置



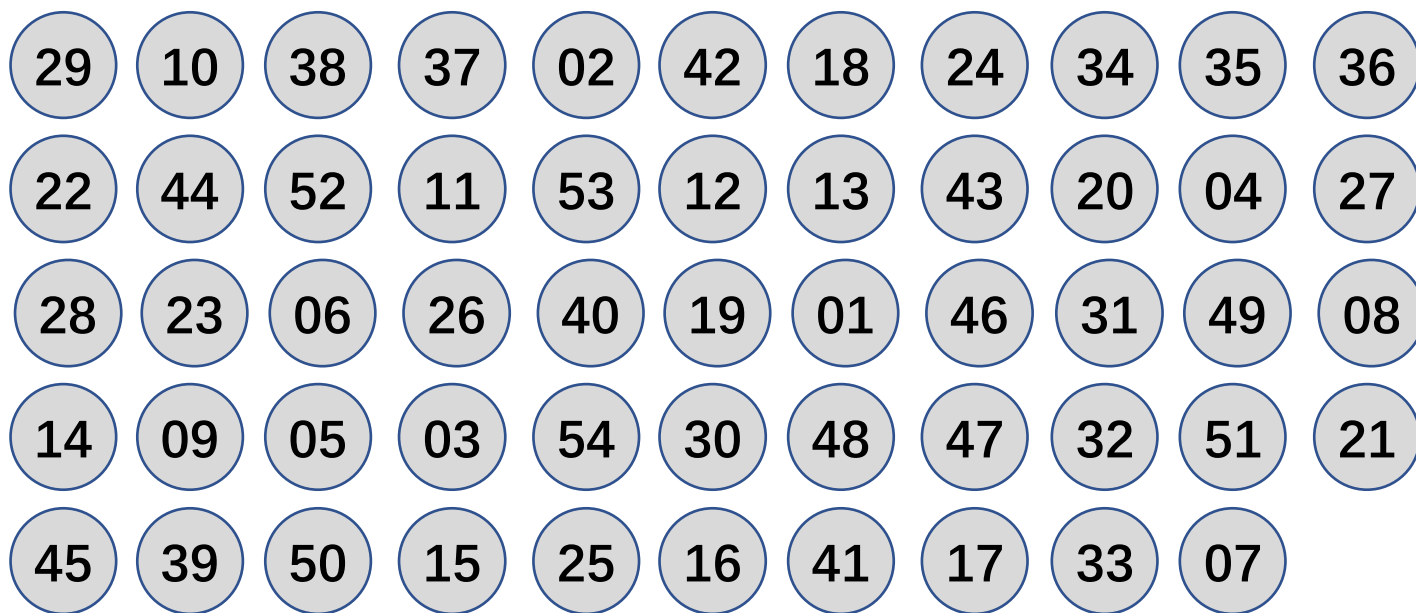
2.9 线性时间选择 实例

$n=54, k=15$

递归初始条件为 $n>5$

Median of Medians, MoM

中位数组中位数



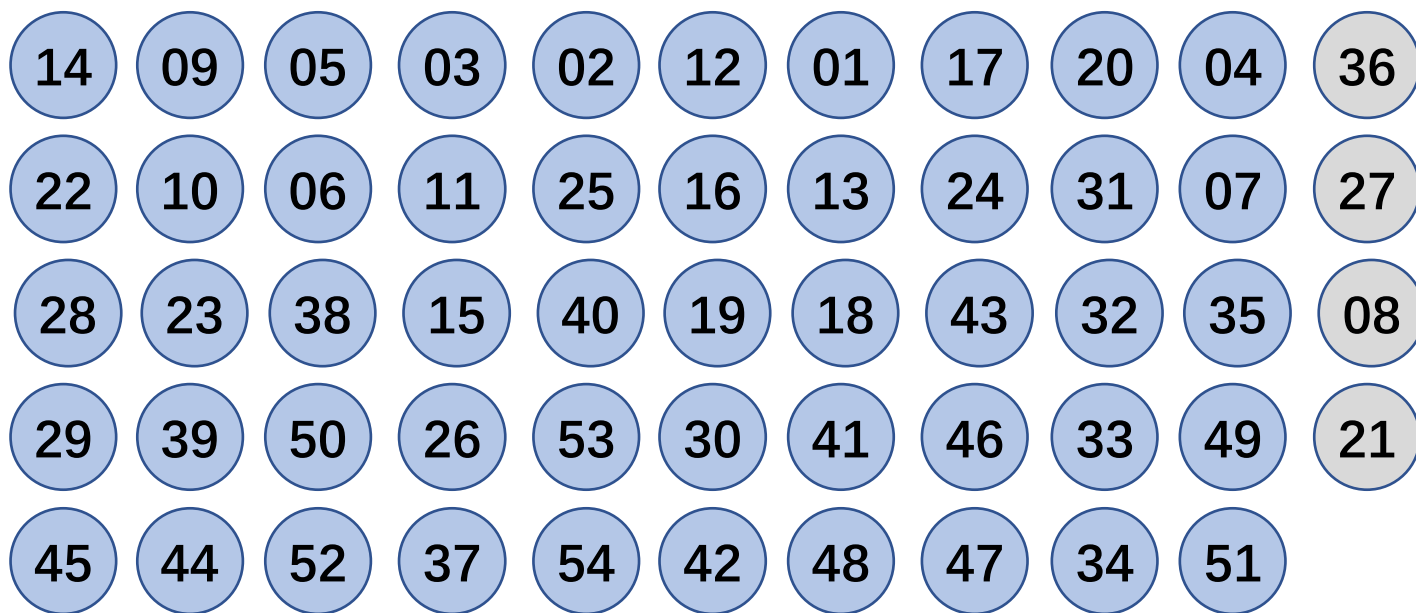
2.9 线性时间选择 实例

$n=54, k=15$

递归初始条件为 $n>5$

Median of Medians, MoM

中位数组中位数



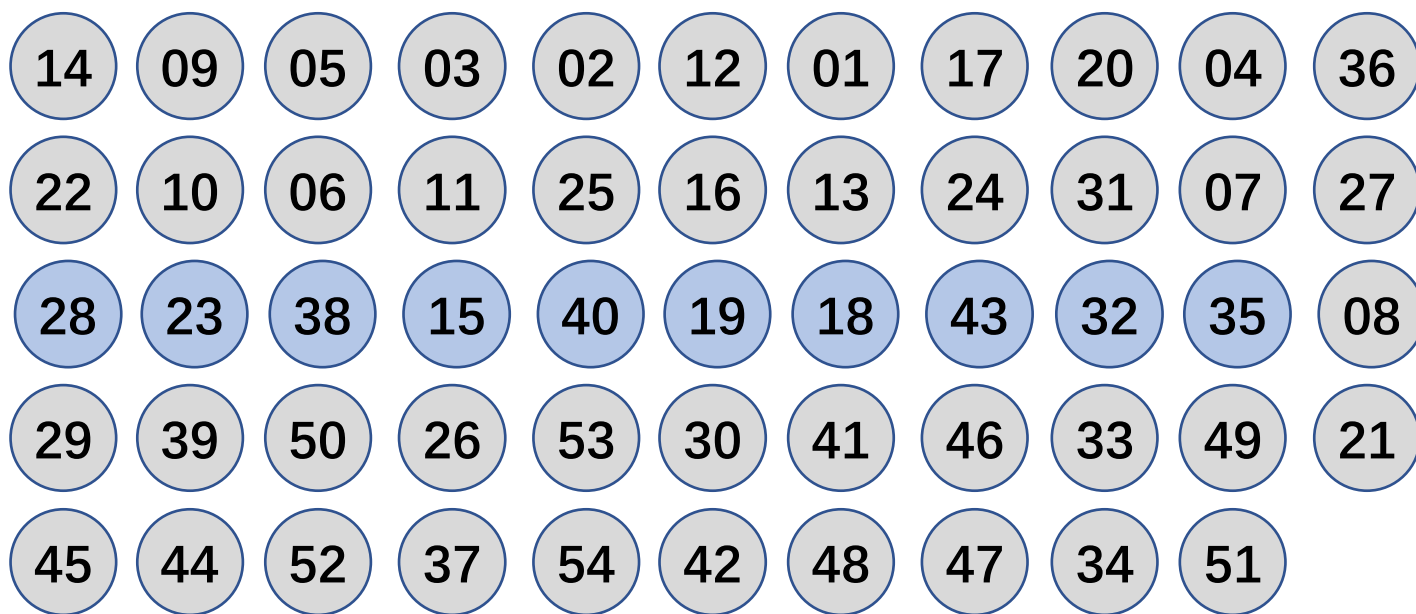
2.9 线性时间选择 实例

$n=54, k=15$

递归初始条件为 $n>5$

Median of Medians, MoM

中位数组中位数



取出每中位数

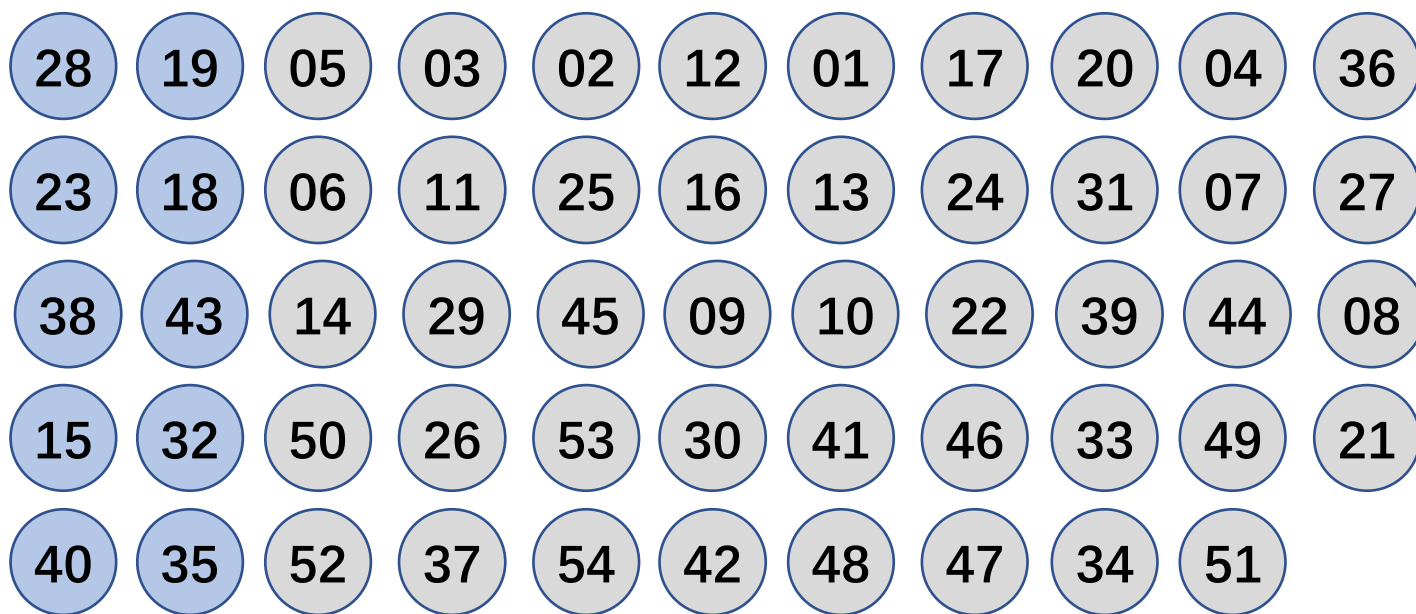
2.9 线性时间选择 实例

$n=54, k=15$

递归初始条件为 $n>5$

Median of Medians, MoM

中位数组中位数



构造中位数数组

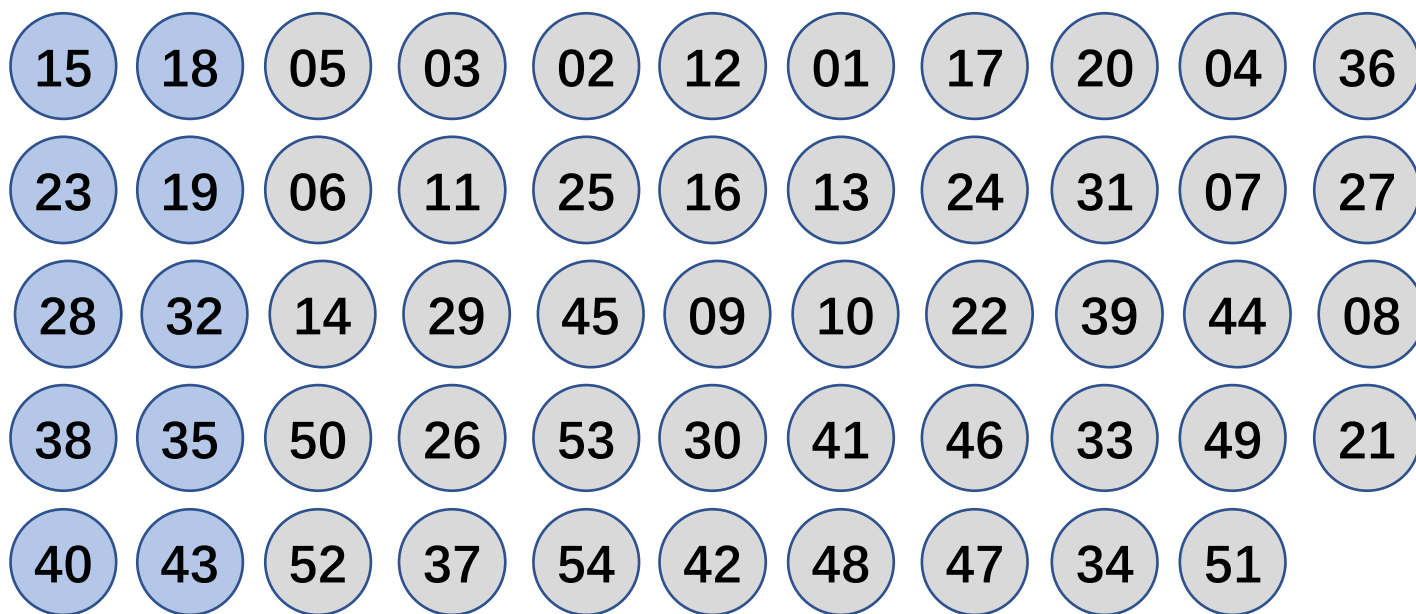
2.9 线性时间选择 实例

$n=54, k=15$

递归初始条件为 $n>5$

Median of Medians, MoM

中位数组中位数



每组 $O(1)$ 排序

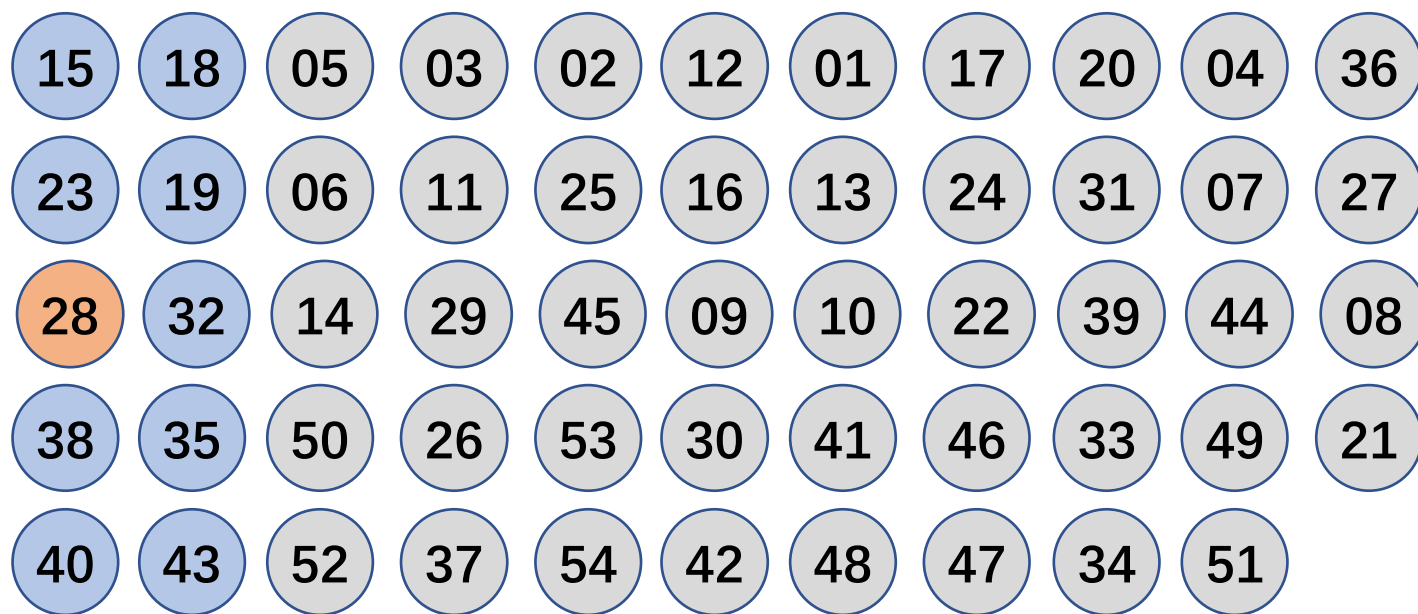
2.9 线性时间选择 实例

$n=54, k=15$

递归初始条件为 $n>5$

Median of Medians, MoM

中位数数组中位数



取出中位数数组[28,32]的中位数28作为
pivot对中位数数组[15,23,...,43]进行partition

2.9 线性时间选择 实例

$n=54, k=15$

递归初始条件为 $n>5$

Median of Medians, MoM

中位数数组中位数

pivot



取出中位数数组[28,32]的中位数28作为
pivot对中位数数组[15,23,...,43]进行partition

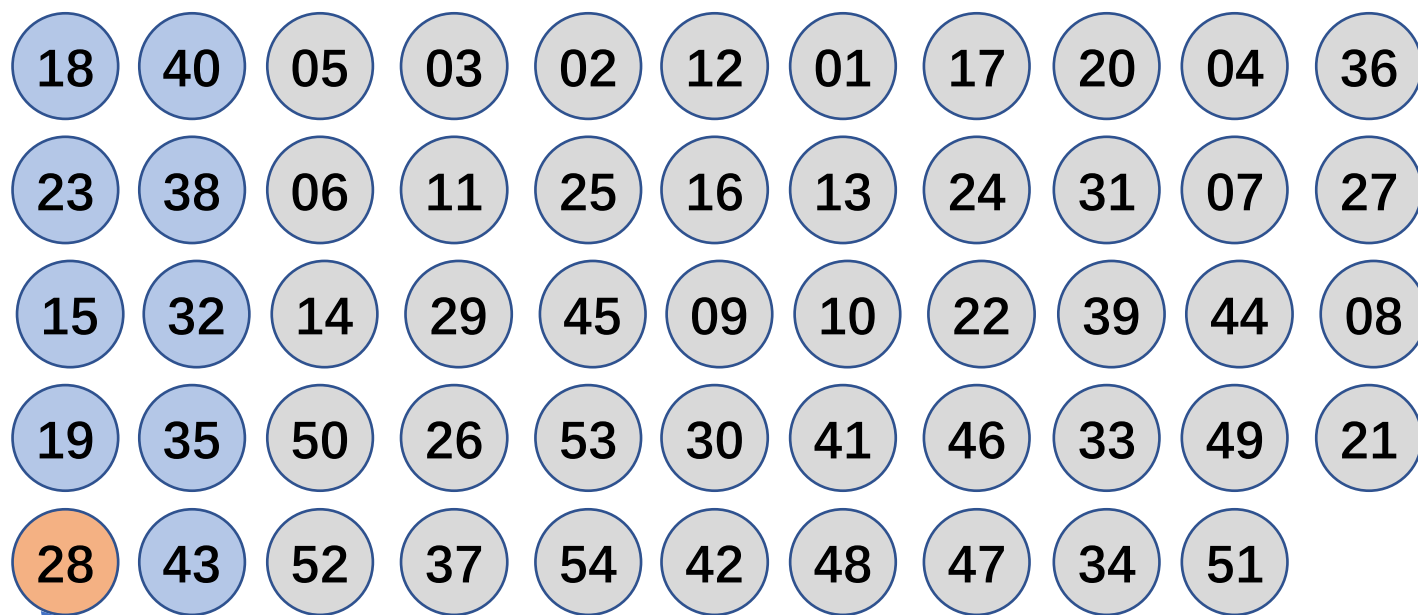
2.9 线性时间选择 实例

$n=54, k=15$

递归初始条件为 $n>5$

Median of Medians, MoM

中位数数组中位数



Median of Medians

取出中位数数组[28,32]的中位数28作为
pivot对中位数数组[15,23,...,43]进行partition

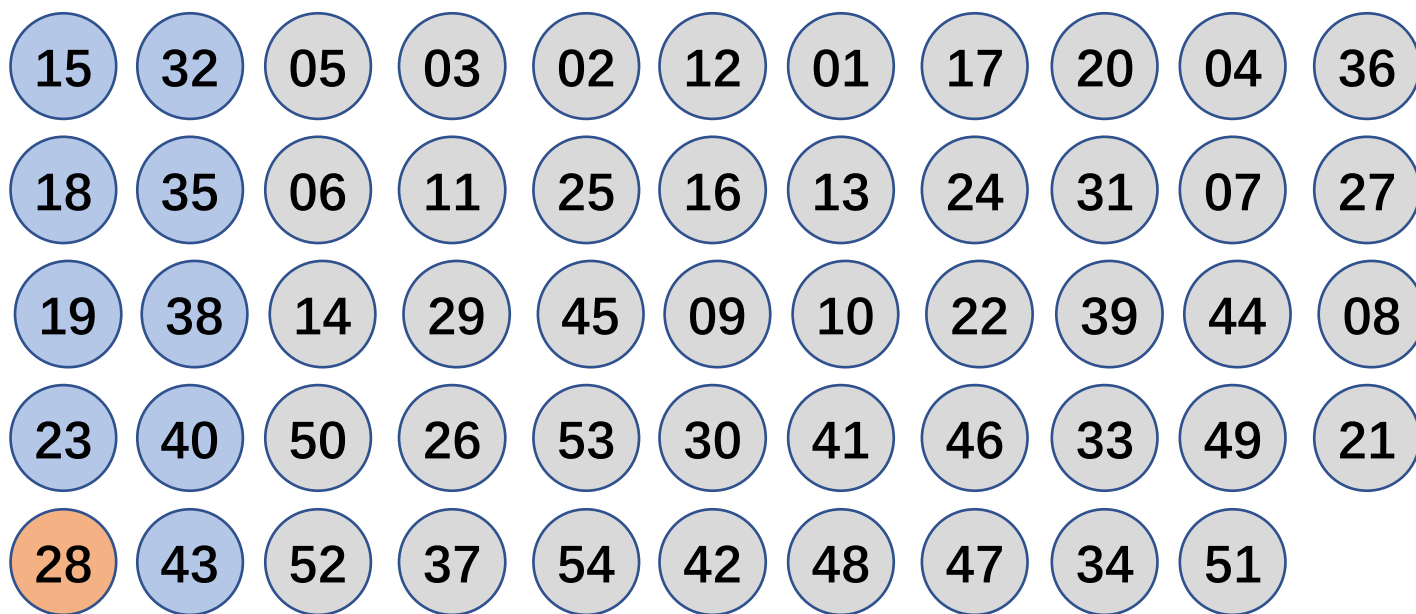
2.9 线性时间选择 实例

$n=54, k=15$

递归初始条件为 $n>5$

Median of Medians, MoM

中位数数组中位数



取出中位数数组[28,32]的中位数28作为
pivot对中位数数组[15,23,...,43]进行partition

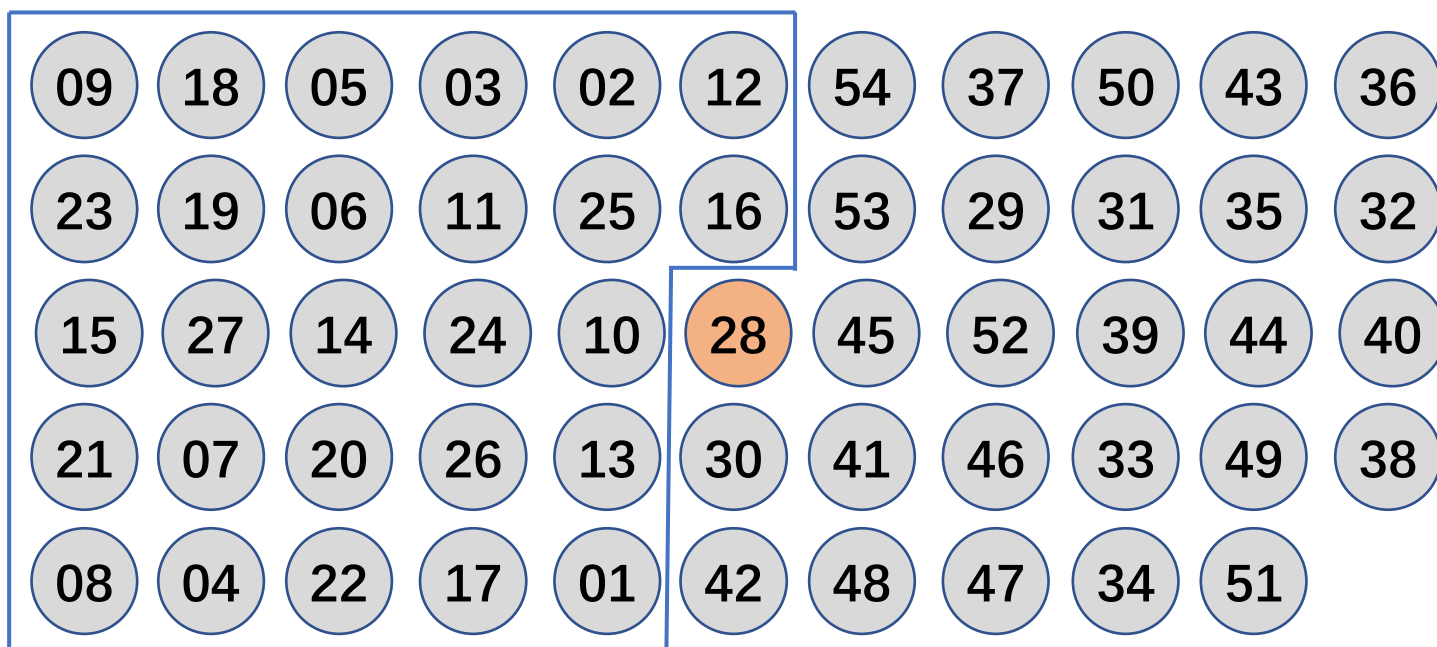
2.9 线性时间选择 实例

$n=54, k=15$

递归初始条件为 $n>5$

Median of Medians, MoM

中位数组中位数



一轮partition后pivot排好序

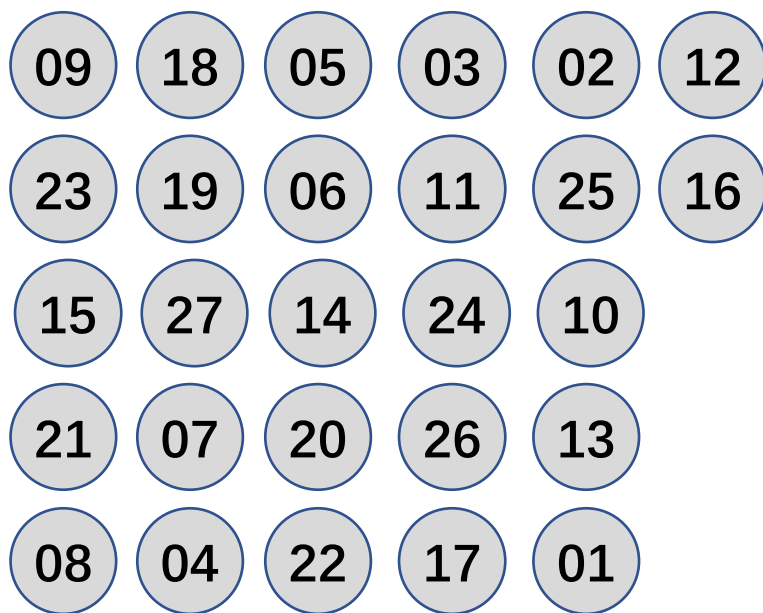
2.9 线性时间选择 实例

$n=54, k=15$

递归初始条件为 $n>5$

Median of Medians, MoM

中位数组中位数



$s > k$

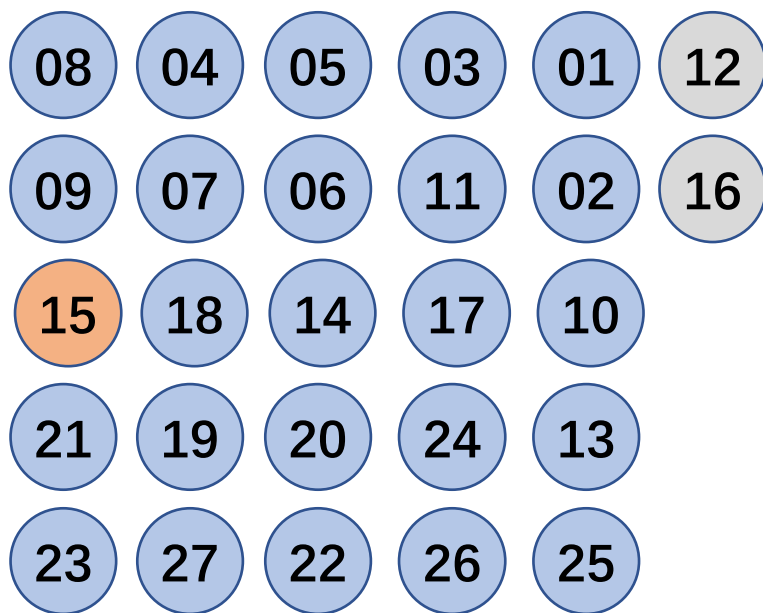
2.9 线性时间选择 实例

$n=54, k=15$

递归初始条件为 $n>5$

Median of Medians, MoM

中位数组中位数



2.9 线性时间选择 实例

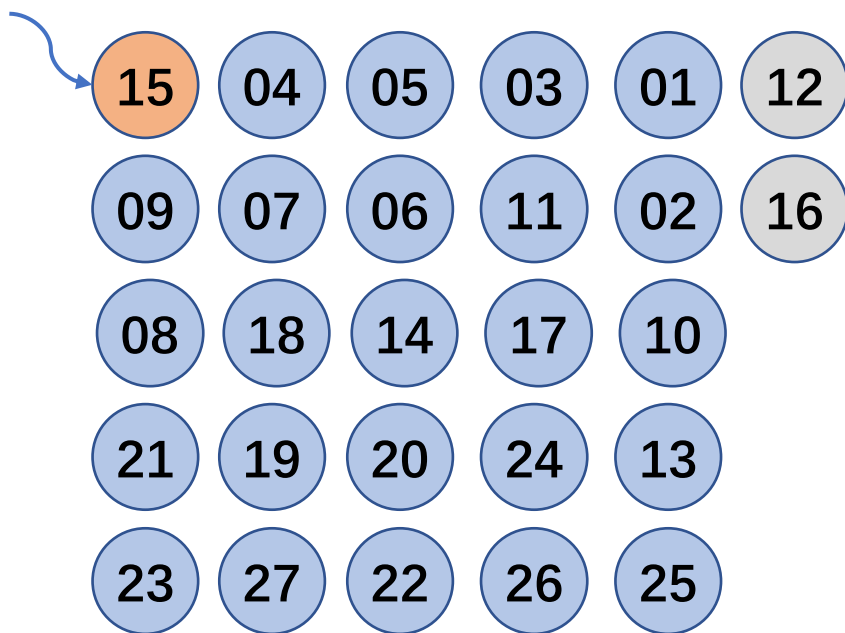
$n=54, k=15$

递归初始条件为 $n>5$

Median of Medians, MoM

中位数组中位数

pivot



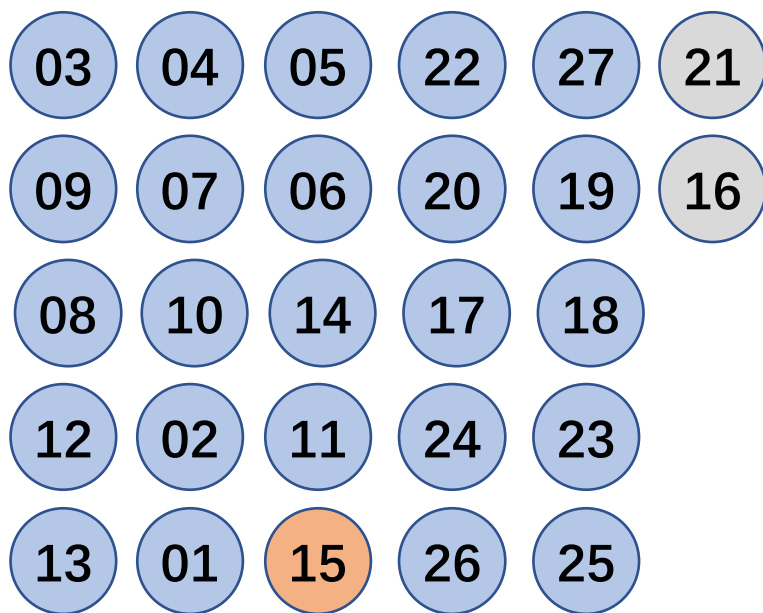
2.9 线性时间选择 实例

$n=54, k=15$

递归初始条件为 $n>5$

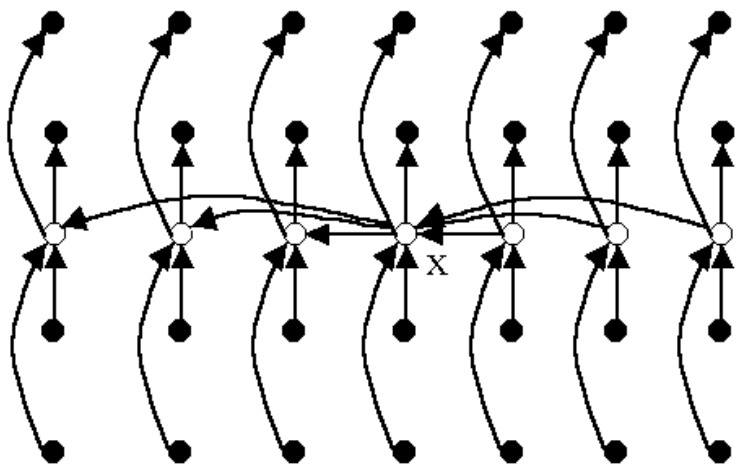
Median of Medians, MoM

中位数组中位数



2.9 线性时间选择

- 将 n 个输入元素划分成 $\lceil n/5 \rceil$ 个组，每组5个元素，只可能有一个组不是5个元素。用任意一种排序算法，将每组中的元素排好序，并取出每组的中位数，共 $\lceil n/5 \rceil$ 个。
- 递归调用select来找出这 $\lceil n/5 \rceil$ 个元素的中位数。如果 $\lceil n/5 \rceil$ 是偶数，就找它的2个中位数中较大的一个。以这个元素作为划分基准。



设所有元素互不相同。在这种情况下，找出的基准 x 至少比 $3(n-5)/10$ 个元素大，因为在每一组中有2个元素小于本组的中位数，而 $n/5$ 个中位数中又有 $(n-5)/10$ 个小于基准 x 。同理，基准 x 也至少比 $3(n-5)/10$ 个元素小。而当 $n \geq 75$ 时， $3(n-5)/10 \geq n/4$ 所以按此基准划分所得的2个子数组的长度都至少缩短 $1/4$ 。

2.9 线性时间选择

Type **Select**(Type a[], int p, int r, int k)

```
{
    if (r-p<75) {
        用某个简单排序算法对数组a[p:r]排序;
        return a[p+k-1];
    };
    for ( int i = 0; i<=(r-p-4)/5; i++ )
        将a[p+5*i]至a[p+5*i+4]的第3小元素
        与a[p+i]交换位置;
    //找中位数的中位数, r-p-4即上面所说的n-5
    Type x = Select(a, p, p+(r-p-4)/5, (r-p-4)/10);
    int i=Partition(a,p,r, x),
    j=i-p+1;
    if (k<=j) return Select(a,p,i,k);
    else return Select(a,i+1,r,k-j);
}
```

上述算法将每一组的大小定为5, 并选取75作为是否作递归调用的分界点。这2点保证了 $T(n)$ 的递归式中2个自变量之和 $n/5 + 3n/4 = 19n/20 = \varepsilon n$, $0 < \varepsilon < 1$ 。这是使 $T(n)=O(n)$ 的关键之处。当然, 除了5和75之外, 还有其他选择。

复杂度分析

$$T(n) \leq \begin{cases} C_1 & n < 75 \\ C_2 n + T(n/5) + T(3n/4) & n \geq 75 \end{cases}$$

$$T(n)=O(n)$$

分治法的复杂性分析

一个分治法将规模为 n 的问题分成 k 个规模为 n/m 的子问题去解。设分解阈值 $n_0=1$ ，且adhoc解规模为1的问题耗费1个单位时间。再设将原问题分解为 k 个子问题以及用merge将 k 个子问题的解合并为原问题的解需用 $f(n)$ 个单位时间。用 $T(n)$ 表示该分治法解规模为 $|P|=n$ 的问题所需的计算时间，则有：

$$T(n) = \begin{cases} O(1) & n = 1 \\ kT(n/m) + f(n) & n > 1 \end{cases}$$

通过迭代法求得方程的解：
$$T(n) = n^{\log_m k} + \sum_{j=0}^{\log_m n - 1} k^j f(n/m^j)$$